Highveld CO_2 Project

Document 2: build tutorial, level by level

Prepared for: Batsirai Mugomba
18 October 2026

What this document is

A guided rebuild. The project is built in 13 levels, each resting on the one before. For every level you get: the goal, why it comes at that point, what each file does (with real code from the repository), a check to run, and the errors you are likely to meet. The complete source is in Appendix D.

How to use it

Type the files yourself in the order given, running the check at the end of each level. Typing forces you to read every line; copy-pasting does not.

If a check fails, fix it **before** moving on. Later levels assume earlier ones work.

If you get stuck, send me the exact error message and the level number. Appendix A lists the common ones.

The code was verified on synthetic data. Level 2 and the reading code in Level 3 touch real NASA files and have **not** been run against real data, so those are where you are most likely to find a genuine surprise.

The map of the build

Read this table first. It shows what depends on what, and why the order is not alphabetical.

Level	Name	You build	Why now
0	Skeleton	folders, `requirements.txt`, `config.py`	Everything imports the configuration
1	Geography and sources	`geo.py`, `build_reference.py`, `sources.py`	Every later step measures distance to a source
2	Getting data	`download.py`, `inspect_file.py`	You need real files, and a way to look inside them
3	Reading and cleaning	`read_oco2.py`, `qc.py`	Raw file to a clean table
4	The test bench	`synthetic.py`	You cannot judge analysis code without a known answer
5	Anomalies	`anomaly.py`	The core idea: background subtraction
6	Events	`events.py`	Turn anomalies into discrete detections
7	Attribution and statistics	`attribute.py`, `composite.py`	Decide whether detections mean anything
8	Figures	`mapping.py`	Show the results honestly
9	Database	`schema.sql`, `db.py`, `docker-compose.yml`	Persist and query spatially
10	Orchestration	`pipeline.py`, `Makefile`	One command runs everything
11	Tests	`tests/`	Lock the behaviour in
12	Notebook, README, publishing	notebook, `README.md`, git	Make it understandable and shareable

Why the test bench (Level 4) comes before the analysis (Levels 5 to 7)

On real data nobody knows the true answer, so a bug in your background estimate would look like a result. By building a generator whose sources, noise and plumes you control, you can check that the analysis recovers what you put in. Build the test bench before the machine.

Level 0: The skeleton and the environment

Goal	A folder structure that keeps code, original data, results and tests apart, and one file where every tunable number lives.
Why now	Every other file imports `config.py`. Keeping raw data separate from code means you never overwrite an original. Putting every analysis choice into one frozen dataclass means a run can be reproduced from its parameters, and a sensitivity test is a one-line change.

Concepts you need

- **Virtual environment:** an isolated set of installed packages for this project, so versions never clash with other work.
- **Frozen dataclass:** a settings object that cannot be modified after creation. You change a setting by making a modified copy with `dataclasses.replace`, which leaves the original intact.
- **Secrets stay out of git:** credentials live in `.env`, which `.gitignore` excludes. Only `.env.example` (blank values) is committed.

The files, one by one

requirements.txt

Every library, with a minimum version.

- `numpy`, `pandas`, `scipy`: arrays, tables, statistics and the KD-tree.
- `scikit-learn`: the DBSCAN clustering used for complexes and hotspots.
- `netCDF4`: reads the satellite files. `pyproj`: converts degrees to metres.
- `geopandas`, `shapely`: tables with geometry. `matplotlib`, `folium`: figures and the interactive map.
- `sqlalchemy`, `geoalchemy2`, `psycopg2-binary`: talk to PostgreSQL/PostGIS.
- `earthaccess`: NASA downloads. `python-dotenv`: reads `.env`. `pytest`, `jupyter`: development.

.gitignore and .env.example

Keep large data, secrets and synthetic output out of the repository.

- `data/raw/oco2/` is large and regenerable, so it is ignored. `.env` holds your NASA password, so it is ignored.
- `results_demo/` is ignored on purpose: synthetic output must never be committed as if it were a result.

highveld_co2/config.py

Paths, the study box, the map projection and every analysis parameter.

- `ROOT = Path(__file__).resolve().parent.parent` finds the repository root from the file's own location, so paths work wherever the project is cloned.
- `BBOX = (27.0, -28.0, 31.0, -24.5)` is west, south, east, north in degrees.
- `METRIC_CRS = "EPSG:32735"` (UTM zone 35S) is where distances are measured in metres.
- `Params` holds ten-odd choices: quality-control modes, pass thresholds, the 40 km exclusion radius, bin length, detection thresholds, attribution radii.
- Read the comment above `attrib_km`: it explains why three radii must be equal. That comment records a mistake made during this build (Appendix B).

```
@dataclass(frozen=True)
```

```
class Params:
    """Analysis parameters. Frozen so a run can be reproduced from its Params."""

    # --- quality control ------------------------------------------------
    keep_modes: tuple = (0, 1)        # operation_mode: 0 nadir, 1 glint (2 target, 3 transition dropped)

    # --- segmenting soundings into overpasses ---------------------------
    pass_gap_s: float = 300.0         # time gap that starts a new pass (a pass over the box lasts ~1 min)
    min_soundings: int = 150          # good soundings needed for a usable pass
    min_far_soundings: int = 60       # far-from-source soundings needed to define a background
    min_extent_km: float = 100.0      # along-track length needed (short clips cannot define a background)

    # --- background and anomaly -----------------------------------------
```

conftest.py and highveld_co2/__init__.py

Two near-empty files that make imports work.

- `__init__.py` marks the folder as a Python package. An empty `conftest.py` in the repository root makes pytest add the root to the import path.

Check that it works

```
python -c "from highveld_co2.config import DEFAULT_PARAMS as P; print(P.exclude_km, P.bin_km)"
```

Expected: Prints `40.0 5.0`.

Common errors

Symptom	Likely cause	Fix
`ModuleNotFoundError: highveld_co2`	You ran Python from the wrong folder, or `__init__.py` is missing.	Run from the repository root; create the empty `__init__.py`.
`pip` installs fail on `netCDF4` or `geopandas`	An old Python or pip.	Upgrade pip (`python -m pip install -U pip`) and use Python 3.10 or newer.

Level 1: Geography and the reference data

Goal	Know where the CO_2 sources are, and measure distances in metres.
Why now	Almost every later question is a distance question: how far is this sounding from a source? So distance machinery comes first, before any satellite data.

Concepts you need

- **Degrees are not distances.** One degree of longitude is about 100 km at 26°S and about 111 km at the equator. To measure distance you project longitude and latitude onto a flat plane in metres (UTM).
- **Axis order.** `pyproj` can swap longitude and latitude. `always_xy=True` forces (longitude, latitude). Forgetting this puts your points on the wrong side of the world.
- **KD-tree:** a data structure that finds the nearest of many points almost instantly. Querying 100,000 soundings against 17 sources through a loop is slow; the tree makes it trivial.
- **Complexes:** the satellite swath is about 10 km wide, so it cannot separate plants a few kilometres apart. Plants closer than 30 km (chained) are grouped into one complex.

The files, one by one

highveld_co2/geo.py

Three small functions: `to_metric`, `to_lonlat`, `haversine_km`.

- `to_metric` turns longitude/latitude into x/y metres; `to_lonlat` reverses it. Both accept scalars or arrays.
- `haversine_km` is the great-circle distance. It is kept only as an independent check: a test confirms the two methods agree to within 0.5 %.

```
_TO_M = Transformer.from_crs("EPSG:4326", METRIC_CRS, always_xy=True)
_TO_LL = Transformer.from_crs(METRIC_CRS, "EPSG:4326", always_xy=True)

EARTH_RADIUS_KM = 6371.0088


def to_metric(lon, lat):
    """lon/lat (degrees) -> x/y in metres in METRIC_CRS. Accepts scalars or arrays."""
    x, y = _TO_M.transform(np.asarray(lon, dtype=float), np.asarray(lat, dtype=float))
```

scripts/build_reference.py

Downloads the WRI Global Power Plant Database and writes `data/reference/power_plants_highveld.csv`.

- Filters to South Africa, fuels Coal/Gas/Oil, at least 100 MW, inside the box plus a 0.5° buffer.
- Result: 16 plants, from Kendal (4,116 MW) to Pretoria West (180 MW).
- Being a script rather than a hand-edited file means anyone can regenerate the list and see exactly how it was made.

data/reference/extra_sources.csv

Sources that are not power plants. Currently Secunda.

- Secunda is a coal-to-liquids plant and a very large CO_2 source, but not in a power-plant database.
- It is added by hand, with a `note` column saying its coordinates are approximate. **Verify them** before relying on them.

highveld_co2/sources.py

Loads the sources, groups them into complexes, and provides fast nearest-source lookups.

- `load_sources` reads both CSVs and adds metric `x_m`, `y_m` columns.
- `group_complexes` runs DBSCAN with `min_samples=1` on the metric coordinates. With one minimum sample, DBSCAN is single-linkage: if A is within 30 km of B and B within 30 km of C, all three join one complex. Complex ids are ordered by total capacity, so id 0 is the biggest.
- `SourceIndex` wraps the KD-tree. Its `query` returns the distance in kilometres and the index of the nearest source for every point.

```
class SourceIndex:
    """Fast 'how far is the nearest source?' lookups in metres, via a KD-tree.

    Building the tree once and querying it for every sounding is what keeps the pipeline
    fast: a naive loop over sources for each of ~100k soundings would be far slower.
    """

    def __init__(self, sources: gpd.GeoDataFrame):
        self.sources = sources.reset_index(drop=True)
        self.tree = cKDTree(self.sources[["x_m", "y_m"]].values)

    def query(self, x_m, y_m):
        """Return (distance_km, source_row_index) of the nearest source for each point."""
        d, i = self.tree.query(np.c_[np.asarray(x_m), np.asarray(y_m)])
        return d / 1000.0, i

    def names(self, idx):
        return self.sources["name"].values[np.asarray(idx)]

    def complex_ids(self, idx):
```

Check that it works

```
python scripts/build_reference.py
python -c "from highveld_co2.sources import *; s,c=group_complexes(load_sources());
print(c[['complex_id','name','n_sources']])"
```

Expected: 16 plants written; then 9 complexes, the largest being the Kendal complex with 5 sources.

Common errors

Symptom	Likely cause	Fix
Points appear thousands of kilometres away; distances are absurd	Longitude and latitude swapped.	Ensure `always_xy=True` and that you pass `(lon, lat)` in that order.
`build_reference.py` fails with an HTTP error	The GitHub path changed or you are offline.	Download the database CSV from the WRI data portal and point the script at the local file.
Only one complex for everything	`eps_km` too large, or km/metres mixed.	DBSCAN's `eps` is in metres: `eps_km * 1000`.

Level 2: Getting the data

Goal	Obtain OCO-2 files from NASA, and be able to look inside any NetCDF file before writing code against it.
Why now	You need real files, and you should never write a reader for a file you have not looked at. The inspection tool also becomes your debugging tool for the whole project.

Concepts you need

- **NetCDF4** stores variables in named groups, with attributes such as units and fill values. A fill value marks “no data” and must be converted to NaN, or it corrupts every average.
- **Earthdata login:** NASA's data need a free account. For GES DISC (where OCO-2 lives) you must also authorise the application **NASA GESDISC DATA ARCHIVE** in your profile, or downloads fail with 401/403 even with a correct password.
- **Daily global files:** each OCO-2 Lite file covers the whole Earth for one day, so a bounding box cannot shrink the download.
- **The 16-day repeat:** the ground track repeats exactly every 16 days. The study box is crossed on only 2 to 3 days of each cycle. Download one cycle, find those days, then fetch only the matching days in later cycles. This cuts the data volume by roughly a factor of six.

The files, one by one

highveld_co2/download.py

Searches and downloads OCO-2 files. Split into pure helper functions (tested) and a thin network part (not testable offline).

- `pick_latest_version`: NASA can return the same day under several product versions. Mixing them double-counts days and mixes algorithms, so exactly one version is kept.
- `cycle_days`: given a cycle start and the day offsets that cross the box, lists every matching date in a range.
- `overpass_offsets`: reads the files from the first cycle and reports which day offsets have soundings inside the box.
- `get` refuses to download more than `--max-files` files, so a typo in a date range cannot start a 100 GB download.

```
def pick_latest_version(granules, wanted=None):
    """Keep only granules of one product version.

    CMR can return the same day under several versions. Mixing them double-counts days and mixes
    retrieval algorithms, so we keep exactly one: `wanted`, or else the numerically newest.
    Returns (granules_kept, version_used).
    """
    def ver(g):
        return g["umm"]["CollectionReference"]["Version"]

    versions = sorted({ver(g) for g in granules}, key=parse_version)
    if not versions:
        return [], None
    use = wanted if wanted else versions[-1]
    return [g for g in granules if ver(g) == use], use
```

highveld_co2/inspect_file.py

Prints the groups, variables, shapes and units of a NetCDF file.

- Run it first on every new file type. If a variable the reader needs has a different name, this is how you find out.

Check that it works

```
python -m highveld_co2.download get --start 2023-06-01 --end 2023-06-16
python -m highveld_co2.inspect_file data/raw/oco2/<one file>.nc4
```

Expected: Up to 16 files download; the inspector lists `xco2`, `xco2_quality_flag`, `latitude` and others, plus a `Sounding` group.

> **Not verified against real NASA data**
> This level was written without access to NASA's servers. The helper functions are unit-tested; the network calls are not. Treat any failure here as expected-in-principle and send me the message.

Common errors

Symptom	Likely cause	Fix
401 or 403 on download	The GES DISC application is not authorised.	Earthdata profile → Applications → Authorized Apps → approve NASA GESDISC DATA ARCHIVE.
Login error or prompt for credentials	`.env` missing or wrong variable names.	Use exactly `EARTHDATA_USERNAME` and `EARTHDATA_PASSWORD`.
CMR returned 0 granules	Wrong dates, or the product short name or version is wrong.	Check the product page on GES DISC; set `OCO2_VERSION` in `config.py`.
`--max-files` error	The date range covers too many days.	Narrow the range, or use the cycle method (`offsets`) to fetch fewer days.

Level 3: Reading and cleaning

Goal	Turn a raw NetCDF file into a clean pandas table of trustworthy soundings.
Why now	Everything downstream assumes tidy, filtered data, so this must be right. The reader is also the only module that knows the file layout, so file-format surprises are fixed in one place.

Concepts you need

- **Masked arrays:** netCDF4 returns masked arrays where fill values are hidden. `np.ma.filled(..., np.nan)` converts masks to NaN, which pandas understands.
- **Quality flag:** `xco2_quality_flag == 0` means NASA considers the sounding good. It removes soundings affected by cloud, aerosol and other problems.
- **Always report what each rule removed.** A rule that silently removes 90 % of the data is a red flag you would otherwise never see.
- **Fail loudly, early, helpfully.** A missing variable raises an error that tells you which command to run next.

The files, one by one

highveld_co2/read_oco2.py

Reads one or many Lite files into a table, keeping only soundings inside the box.

- `REQUIRED` and `OPTIONAL` map our column names to paths inside the file (for example `"op_mode": "Sounding/operation_mode"`). If a variable is named differently in your file version, you change that one dictionary.
- `_get` walks into groups, returns a float array with NaN for fill values, or `None` if the variable is absent.
- Time arrives as seconds since 1970 and is converted to timezone-aware datetimes.
- `read_many` refuses to mix synthetic and real files: a mixed run would silently corrupt results.

```
def _get(ds: Dataset, path: str):
    """Fetch one variable as a float array with NaN for fill values; None if absent."""
    node = ds
    *groups, var = path.split("/")
    for g in groups:
        if g not in node.groups:
            return None
        node = node.groups[g]
    if var not in node.variables:
        return None
    arr = node.variables[var][:]
    return np.ma.filled(np.ma.asarray(arr).astype("float64"), np.nan)
```

highveld_co2/qc.py

Applies three filters and returns a report of what each removed.

- Rule 1: quality flag equals 0. Rule 2: land only. Rule 3: operation mode in nadir or glint (target and transition modes have unusual geometry).
- A rule whose column is entirely missing is skipped and noted in the report, rather than dropping everything.

```
    n0 = len(df)
    report = {"n_in": n0}
```

```
    ok = df["qf"] == 0
    report["removed_quality_flag"] = int((~ok).sum())

    if df["land_water"].notna().any():
        land = df["land_water"].fillna(0) == 0
        report["removed_water"] = int((ok & ~land).sum())
        ok &= land
    else:
        report["land_water_rule"] = "skipped (column missing)"

    if df["op_mode"].notna().any():
        mode_ok = df["op_mode"].isin(params.keep_modes) | df["op_mode"].isna()
        report["removed_operation_mode"] = int((ok & ~mode_ok).sum())
        ok &= mode_ok
    else:
        report["operation_mode_rule"] = "skipped (column missing)"
```

Check that it works

```
python - <<'EOF'
from pathlib import Path
from highveld_co2.read_oco2 import read_many
from highveld_co2.qc import apply_qc
df = read_many(Path('data/raw/oco2').glob('*.nc4'))
clean, rep = apply_qc(df); print(rep)
EOF
```

Expected: A report with `n_in`, `n_out`, and how many each rule removed. On real data a large share can be removed. Look at the numbers; do not assume.

Common errors

Symptom	Likely cause	Fix
`KeyError: required variable ... not found`	Your file version names the variable differently.	Run `inspect_file`; edit `REQUIRED`.
Every sounding removed	The flag semantics differ, or the column is all NaN.	Print `df['qf'].value_counts()`; check the file's documentation.
Dates in 1970	Time units differ from seconds since 1970.	Check the `units` attribute of `time` in `inspect_file`.
`ValueError: Refusing to mix synthetic and real files`	Synthetic and real files are in one folder.	Keep them in separate folders (`data/raw/synthetic` and `data/raw/oco2`).

Level 4: The test bench: synthetic OCO-2 data

Goal	A generator that writes OCO-2-format files containing a known truth, so the analysis can be tested.
Why now	On real data you cannot tell a correct algorithm from a buggy one that produces plausible output. With synthetic data you know exactly where the plumes are.

Concepts you need

- **Simulate what makes the problem hard**, not just the signal: realistic sampling, noise that is correlated along the track, cloud, and artefacts that are not sources. Early versions of this generator were too clean and produced almost no false alarms, which made the control test meaningless (Appendix B).
- **Seeds:** `np.random.default_rng(seed)` makes the “random” data identical on every run, which is what allows deterministic tests.
- **Honesty stamps:** synthetic files carry a `synthetic = "true"` attribute; the pipeline prints a banner, stamps figures, and refuses to mix them with real data.

The files, one by one

highveld_co2/synthetic.py

Writes one NetCDF file per overpass day, plus a truth table.

- **Sampling:** three ground tracks in a 16-day cycle, eight footprints across a 10 km swath, a row every 2.25 km.
- **Background:** a trend, a seasonal cycle, a per-pass offset, an along-track gradient, spatially correlated error (0.5 ppm, 25 km correlation length) and white noise tied to the reported uncertainty.
- **Plumes:** from the real plant locations. Each is a Gaussian across the wind direction that decays exponentially downwind (40 km scale), with amplitude scaled by capacity, operating factor and inverse wind speed.
- **Traps:** two urban domes (Johannesburg, Pretoria) that sit near small plants, and localised retrieval artefacts in about 12 % of passes.
- **Cloud:** patchy bad-quality soundings; about one pass in ten is almost fully clouded.

```
def _plume_enhancement(x, y, src, wind_dir_deg, wind_ms, rng, amplitude_scale):
    """Sum of Gaussian-like plumes (ppm) at points (x, y) in metres; returns (enh, truth rows)."""
    enh = np.zeros_like(x)
    truth = []
    for r in src.itertuples():
        cap = 4000.0 if pd.isna(r.capacity_mw) else float(r.capacity_mw)
        operating = rng.uniform(0.5, 1.0)
        theta = np.radians(wind_dir_deg + rng.normal(0, 15))
        amp = min(4.0, 2.0 * (cap / 4000.0) ** 0.8 * operating * (4.0 / wind_ms) * amplitude_scale)
        dx, dy = x - r.x_m, y - r.y_m
        wx, wy = np.sin(theta), np.cos(theta)
        d = (dx * wx + dy * wy) / 1000.0        # km downwind
        c = (dx * wy - dy * wx) / 1000.0        # km crosswind
        sy = 2.5 + 0.12 * np.clip(d, 0, None)
        shape = np.where(d > 0, np.exp(-0.5 * (c / sy) ** 2) * np.exp(-d / 40.0) * (1 - np.exp(-d / 4.0)),
0.0)
        enh += amp * shape
        truth.append({"source": r.name, "amplitude_ppm": round(amp, 3),
```

```
                      "wind_dir_deg": round(float(np.degrees(theta)) % 360, 1), "wind_ms": round(wind_ms,
2)})
    return enh, truth


def _urban(x, y):
```

Check that it works

```
python -m highveld_co2.synthetic --days 730
```

Expected: `Wrote 137 synthetic overpass files` into `data/raw/synthetic`, plus `synthetic_truth.csv`.

Common errors

Symptom	Likely cause	Fix
NetCDF error writing `int64`	The classic NetCDF format has no 64-bit integers.	Create the file with `format="NETCDF4"`.
Files written but reader finds nothing	Track longitudes fall outside the box.	Check `TRACKS` against `BBOX`.

Level 5: Anomalies: background subtraction

Goal	For every overpass, estimate the background XCO_2 and subtract it to leave the local enhancement.
Why now	This is the central idea of the project. Raw XCO_2 is dominated by a 415 ppm background that varies from day to day; the 1 to 2 ppm plume is invisible until you remove it.

Concepts you need

- **Overpass segmentation:** sort soundings by time and start a new pass at any gap over 5 minutes. A pass over the box lasts about a minute; orbits are about 99 minutes apart.
- **Along-track coordinate:** the footprints of one pass form a thin strip. The first principal axis of their x/y positions is the along-track direction, so projecting each sounding onto it gives one number: distance along the track. Sorting by time instead would zigzag across the eight footprints. (Computed with `numpy.linalg.svd`.)
- **Far-field background:** use only soundings more than 40 km from any known source. Otherwise the plume would be part of its own background.
- **Robust statistics:** the median ignores outliers; the median absolute deviation times 1.4826 estimates the standard deviation without being inflated by them.
- **Running median:** the median in consecutive 100 km windows along the track, interpolated between window centres. It follows slow trends that a single number would leave in the anomaly.

The files, one by one

highveld_co2/anomaly.py

Five functions that take soundings to anomalies.

- `assign_passes` labels each sounding `YYYY-MM-DD_n`.
- `along_track_km` computes the along-track coordinate and orients it so it increases with time.
- `estimate_background` returns the background at every sounding plus a robust scatter, floored at 0.3 ppm so a quiet pass never gets an implausibly small noise estimate.
- `compute_anomalies` loops over passes. A pass is valid only if it has enough good soundings (150), enough far-field soundings (60) and enough extent (100 km). It returns the valid soundings and a table of all passes.

```
def estimate_background(s_km, xco2, far_mask, method: str, window_km: float, sigma_floor: float):
    """Background value at every sounding of one pass, plus a robust scatter.

    method="median":  one number, the median XCO2 of the far soundings.
    method="running": the median in consecutive `window_km` windows along the track, linearly
                      interpolated. This follows slow along-track trends (a real gradient or a
                      retrieval artefact) that a single number would leave in the anomaly.
    Scatter sigma = 1.4826 * MAD of the far soundings about the background (robust to the odd
    plume that leaks into the far field), floored at `sigma_floor`.
    """
    far_x = xco2[far_mask]
    bg_med = np.median(far_x)
    bg = np.full(len(xco2), bg_med)
    if method == "running":
        edges = np.arange(s_km.min(), s_km.max() + window_km, window_km)
        centers, meds = [], []
        for lo, hi in zip(edges[:-1], edges[1:]):
            m = far_mask & (s_km >= lo) & (s_km < hi)
```

```
            if m.sum() >= 10:
                centers.append(s_km[m].mean())
                meds.append(np.median(xco2[m]))
        if len(centers) >= 2:
            bg = np.interp(s_km, centers, meds)   # flat beyond the first/last centre
    sigma = max(sigma_floor, MAD_TO_SIGMA * np.median(np.abs(far_x - bg[far_mask])))
    return bg, float(sigma)


def compute_anomalies(df: pd.DataFrame, index: SourceIndex, params: Params = DEFAULT_PARAMS):
```

Check that it works

```
python -m pytest tests/test_anomaly.py -q
```

Expected: 5 passed. The tests include a known-answer check: a running background follows a planted gradient (error < 0.3 ppm) while a single median cannot (error > 1 ppm).

Common errors

Symptom	Likely cause	Fix
Every pass invalid	Thresholds too strict, or sources cover the whole box so no far-field remains.	Lower `min_far_soundings` or `exclude_km` in `Params`; print `passes[['n_good','n_far','extent_km']]`.
Anomalies are almost all positive	The plume leaked into the background.	Increase `exclude_km`, or use the median method.
Noise estimate stuck at 0.30	The floor is being hit.	Check the scatter of the far field directly; look for bad data.

Level 6: Events: from anomalies to detections

Goal	Turn the anomaly field into discrete, countable events.
Why now	You need a rule that decides “this is a plume”. It has to be strict enough that noise does not trigger it and loose enough that real plumes do.

Concepts you need

- **Averaging reduces noise by √n.** A 5 km bin holds roughly 10 to 40 soundings, so its mean is much steadier than any single sounding.
- **Two conditions, not one.** A bin is flagged if its mean anomaly is at least 1 ppm (it matters) **and** at least 3 standard errors above zero (it is not noise).
- **Runs of consecutive bins:** a new event starts whenever the pass changes or the bin number is not exactly one more than the last flagged bin. The trick is a cumulative sum of “this starts a new run”.
- **The standard error assumes independent soundings.** Real errors are correlated, so it is optimistic. That is why Level 7 measures the false-alarm rate empirically instead of trusting the z-score.

The files, one by one

highveld_co2/events.py

`bin_passes` builds the bin table; `group_events` merges flagged runs.

- `bin_passes` groups soundings by pass and bin, computes the mean anomaly, standard error, z-score and the distance to the nearest source, and sets the `flagged` column.
- `group_events` returns one row per event: length, peak anomaly, peak z, and a position weighted by anomaly size.

```
    new_run = (f["pass_id"] != f["pass_id"].shift()) | (f["bin_idx"] - f["bin_idx"].shift() != 1)
    f["run"] = new_run.cumsum() - 1
    rows = []
    for run, g in f.groupby("run"):
```

Check that it works

```
python -m pytest tests/test_events_stats.py -q -k "plume or noise or minimum"
```

Expected: A planted 4-bin plume becomes exactly one event; 30 passes of pure noise produce at most 3 flagged bins; bins with too few soundings are dropped.

Common errors

Symptom	Likely cause	Fix
Hundreds of events everywhere	Thresholds too loose, or the noise estimate too small.	Raise `z_bin` or `min_enh_ppm`; compare `sigma_ppm` with the reported uncertainty.
No events at all	Thresholds too strict, or `min_bin_n` too high for sparse bins.	Lower `min_enh_ppm`; print the distribution of `z`.

Level 7: Attribution and statistics

Goal	Decide whether detections are real, link them to sources, and say how often each source is seen.
Why now	A list of events is not a result. The result is the answer to “do events cluster near sources more than chance, and how often is each source detected when observed?”

Concepts you need

- **Detection frequency, not detection count.** Because every sounding is kept, we can count the passes that observed each complex and report events divided by observations.
- **Wilson interval:** a 95 % confidence interval for a proportion that behaves well for small counts and for zero events.
- **Control test:** the flag rate near sources against the rate far from them. Flags far from any source are false alarms, so this measures the method's own noise.
- **Bootstrap over passes:** resample whole overpasses (not bins) with replacement many times. Bins within a pass share a background, so only passes are independent.
- **Fisher's exact test:** tests whether the near rate exceeds the far rate. With thousands of bins its p-value can underflow to exactly zero, so it is clamped to a tiny positive number rather than reported as impossible.
- **One radius:** attribution, the near zone and the coverage radius all equal the 40 km exclusion radius (see Appendix B for why).

The files, one by one

highveld_co2/attribute.py

Attribution, hotspots, coverage statistics and the control test.

- `wilson_ci(k, n)` is a nine-line function you can check by hand (0 events in 10 passes gives an upper bound of 27.7 %).
- `link_events` adds nearest source, distance, complex and an `attributed` flag.
- `cluster_hotspots` groups events within 20 km with DBSCAN and reports whether a source explains each.
- `complex_stats` counts, per complex, passes with at least 2 valid bins within 40 km (observed) and passes with at least one flagged bin (event).
- `control_test` returns the near and far rates, their ratio, a Fisher p-value and a bootstrap interval for the ratio.

```
def wilson_ci(k: int, n: int, z: float = 1.96):
    """95 % Wilson score interval for a proportion k/n. Sensible even for small n or k=0."""
    if n == 0:
        return (np.nan, np.nan)
    p = k / n
    denom = 1 + z ** 2 / n
    centre = (p + z ** 2 / (2 * n)) / denom
    half = z * np.sqrt(p * (1 - p) / n + z ** 2 / (4 * n ** 2)) / denom
    return (max(0.0, centre - half), min(1.0, centre + half))
```

highveld_co2/composite.py

Averages many sparse passes into a map grid and a distance profile.

- `grid_mean` bins anomalies into 0.1° cells and drops cells with fewer than 5 bins.

- `radial_profile` computes the mean anomaly against distance to the nearest source, with a bootstrap-over-passes interval. Beyond the exclusion radius it is near zero by construction; the information is in the inner rings.

```
def radial_profile(bins: pd.DataFrame, edges_km=(0, 10, 20, 30, 40, 60, 80, 100), n_boot: int = 2000,
seed: int = 0):
    """Mean anomaly vs distance to the nearest source, with a bootstrap-over-passes 95 % CI.

    Honest reading: the background was built from soundings > exclude_km from any source, so
    rings beyond that distance are ~0 BY CONSTRUCTION. The information is in the inner rings.
    """
    edges = np.asarray(edges_km, dtype=float)
    ring = np.digitize(bins["dist_source_km"].values, edges) - 1
    ok = (ring >= 0) & (ring < len(edges) - 1)
    b = bins.loc[ok].assign(ring=ring[ok])
    passes = b["pass_id"].unique()
    pidx = {p: i for i, p in enumerate(passes)}
    n_r = len(edges) - 1
    sums = np.zeros((len(passes), n_r))
```

Check that it works

```
python -m pytest tests/test_events_stats.py -q
```

Expected: All pass, including: Wilson intervals match hand calculations; the control test finds planted enrichment; and, with no enrichment, p < 0.01 occurs in at most 3 of 20 runs.

Common errors

Symptom	Likely cause	Fix
`rate_ratio` is NaN	No flagged bins in the far zone, so the ratio divides by zero.	That is a good sign, not a bug; report the near rate and a bound.
Complex shows 0 observed passes	No overpass came within the coverage radius.	That is a real finding about coverage; report it, do not hide it.
Pandas warning from `groupby.apply`	An old pattern needing pandas 2.2+.	The code uses a plain `groupby().sum()` instead, which works on every version.

Level 8: Figures and the interactive map

Goal	Four static figures and one interactive map that show the results without misleading.
Why now	Figures are what people actually look at. They come after the numbers exist, and they follow explicit rules.

Concepts you need

- **Polarity gets a diverging colour scale** with a neutral grey midpoint, so zero reads as “nothing” (blue for negative, red for positive).
- **One series, one colour, no legend box.** Several series get a legend and a shape difference, so identity never depends on colour alone.
- **Text in ink colours,** thin marks and a quiet grid.
- **Stamp synthetic figures** `SYNTHETIC DATA`.
- **Show what was not seen.** The anomaly map leaves unsampled cells blank, which makes the satellite's coverage gaps visible.
- **Always look at the figure.** The control chart's label first collided with its title; only viewing the image revealed it (Appendix B).

The files, one by one

highveld_co2/mapping.py

Five output functions.

- `fig_anomaly_map`: mean anomaly per 0.1° cell with sources overlaid.
- `fig_radial_profile`: mean anomaly against distance with a 95 % band.
- `fig_example_pass`: one overpass, with single soundings, bin means, flagged bins, and the distance to the nearest source underneath.
- `fig_control`: near against far flag rates with Wilson intervals, with headroom so labels never hit the title.
- `interactive_map`: a Folium HTML map with sources, events and hotspots as toggleable layers.

```
DIV = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#2a78d6", "#9ec5f4", NEUTRAL, "#f3b0ad", "#e34948", "#a02a2a"])
```

Check that it works

```
make demo     # then open results_demo/fig_*.png and results_demo/map.html
```

Expected: Four PNGs, each stamped SYNTHETIC DATA, and a map you can zoom. Check labels, overlaps and empty areas by eye.

Common errors

Symptom	Likely cause	Fix
Map is blank	Grid indices fall outside the array, or no bins pass the 5-bin minimum.	Print `grid.describe()`; lower `min_count`.

Symptom	Likely cause	Fix
Basemap tiles fail	Some tile providers now need an API key.	This project uses OpenStreetMap tiles, which need none.

Level 9: The database

Goal	Store every result in PostGIS and query it spatially.
Why now	A database lets anyone ask new questions of the results without re-running Python, and shows you can design a spatial schema.

Concepts you need

- **PostGIS** adds geometry types and spatial functions to PostgreSQL. `GEOMETRY(Point, 4326)` stores a longitude/latitude point.
- **GiST index:** the spatial index that makes “within 40 km of this point” fast.
- **`geometry` vs `geography`:** casting to `geography` makes distances come out in true metres on the ellipsoid, which is what `ST_DWithin(..., 40000)` needs.
- **Parents first.** Tables with foreign keys must be created, and loaded, after the tables they reference. Truncating uses `CASCADE`.
- **NaN is not NULL.** Pandas writes NaN as the value `'NaN'`, which silently breaks averages and `IS NULL`. Convert NaN to None first.
- **Idempotent load:** truncate then insert, so re-running never duplicates rows.

The files, one by one

sql/schema.sql

Six tables and five indexes.

- `source_complexes` and `sources`, then `passes`, `bins`, `events` and `complex_stats`, each with the right keys.
- Partial index `bins_flag_ix ... WHERE flagged` makes queries on the (few) flagged bins cheap.

```
CREATE TABLE IF NOT EXISTS events (
    event_id          INTEGER PRIMARY KEY,
    pass_id           TEXT NOT NULL REFERENCES passes(pass_id),
    obs_date          DATE NOT NULL,
    n_bins            INTEGER,
    length_km         DOUBLE PRECISION,
    peak_anomaly_ppm  DOUBLE PRECISION,
    mean_anomaly_ppm  DOUBLE PRECISION,
    peak_z            DOUBLE PRECISION,
    nearest_source    TEXT,
    dist_source_km    DOUBLE PRECISION,
    attributed        BOOLEAN,
    complex_id        INTEGER REFERENCES source_complexes(complex_id),
    hotspot_id        INTEGER,
    geom              GEOMETRY(Point, 4326) NOT NULL
);
```

docker-compose.yml

Starts a PostGIS 16 container with user, password and database all named `co2`.

- `docker compose up -d` starts it; `docker compose down` stops it. Data persists in a named volume.

highveld_co2/db.py

Connects, creates the schema, loads every table and holds five example queries.

- `_null_safe` converts NaN to None and numpy integers to Python integers, which the database driver can adapt.
- `save_all` creates the schema, truncates, then loads in parent-first order, and returns row counts.
- `EXAMPLE_QUERIES` includes detection frequency, events within a distance of a point, unattributed events, monthly counts, and the control test in SQL. The control test takes `:near` and `:far` parameters whose defaults come from `Params`, so SQL and Python agree.

```
def _null_safe(df: pd.DataFrame) -> pd.DataFrame:
    """NaN -> None for every non-geometry column.

    Without this, NaN floats are written as the value 'NaN' (not NULL), which silently breaks
    AVG(), COUNT(col) and any 'IS NULL' test. astype(object) also turns numpy ints/bools into
    plain Python values, which the database driver can adapt.
    """
    out = df.copy()
    for c in out.columns:
        if c == "geom":
            continue
        out[c] = out[c].astype(object).where(out[c].notna(), None)
    return out
```

Check that it works

```
docker compose up -d
python -m highveld_co2.pipeline --raw data/raw/synthetic --out results_demo --db
```

Expected: `rows: {'source_complexes': 9, 'sources': 17, 'passes': 137, 'bins': 6992, 'events': 167, 'complex_stats': 9}` (for the 730-day synthetic set). Running it twice gives the same counts.

Common errors

Symptom	Likely cause	Fix
`connection refused`	The container is not running or the port is taken.	`docker compose ps`; check `DATABASE_URL` in `.env`.
`type "geometry" does not exist`	The PostGIS extension is not enabled.	Use the `postgis/postgis` image, or run `CREATE EXTENSION postgis;`.
`can't adapt type 'numpy.int64'`	NumPy values reached the driver.	Pass data through `_null_safe` (the `astype(object)` step does the conversion).
Foreign-key violation on load	Tables loaded in the wrong order.	Load parents first: complexes, sources, passes, bins, events, stats.

Level 10: Orchestration: one command runs everything

Goal	A single entry point that runs all eight stages, writes every output, and returns the intermediate tables for the notebook.
Why now	Last of the code, because it only wires together what already works. A single entry point is what makes a project reproducible.

Concepts you need

- **`run()` returns a dictionary** of every intermediate table, so the notebook and tests can reuse the same code path as the command line.
- **Provenance:** `summary.json` records the file count, date range, quality-control report, noise estimates, control-test results, and the exact `Params` used.
- **Fail with advice:** if no files or no valid passes exist, the error says what to try.
- **Parameter overrides from the command line** (for example `--exclude-km 60`) make sensitivity runs one-liners.

The files, one by one

highveld_co2/pipeline.py

`run()` plus a command-line wrapper.

- Prints a banner if the data are synthetic.
- Writes `passes.csv`, `bins.csv.gz`, `events.csv`, `hotspots.csv`, `complex_stats.csv`, `radial_profile.csv`, `summary.json`, four PNGs and `map.html`.
- Loads PostGIS only if `--db` is given, importing the database module lazily so the pipeline runs without database libraries configured.

```
def run(raw_dir=RAW_DIR, out_dir=RESULTS_DIR, params: Params = DEFAULT_PARAMS, use_db=False,
        make_figures=True, verbose=True) -> dict:
    """Execute every stage and return all intermediate tables (the notebook uses this too)."""
    log = print if verbose else (lambda *a, **k: None)
    raw_dir, out_dir = Path(raw_dir), Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    files = sorted(list(raw_dir.glob("*.nc4")) + list(raw_dir.glob("*.nc")))
```

Makefile

Short names for common commands.

- `make test`, `make demo` (synthetic), `make real` (real data into `results/`), `make db-up`, `make clean`.

Check that it works

```
make demo
```

Expected: Eight numbered stages print, ending `done -> results_demo`, in roughly ten seconds, with a SYNTHETIC DATA banner.

Common errors

Symptom	Likely cause	Fix
`No .nc/.nc4 files in ...`	Wrong `--raw` folder.	Check the path; synthetic files are in `data/raw/synthetic`.
`No valid overpasses`	Thresholds too strict for your data.	Loosen `min_soundings` and `min_extent_km`.

Level 11: Tests

Goal	Automatic checks that lock the behaviour in, so a later change cannot silently break it.
Why now	Written after the code here so you can see what is being protected. In your own projects, writing a test straight after each function is even better.

Concepts you need

- **Known-answer tests:** build data where you know the answer (a plume of 2 ppm in four bins) and check the code finds it.
- **Calibration tests:** with no real effect, a significance test should say “significant” about as often as its level promises. A single seed would fail by chance, so run many.
- **Integration tests** run the whole pipeline on a small synthetic set; **regression tests** pin a bug that once happened (the SQL control test must equal the Python one).
- **Safe database test:** it only runs if you set `TEST_DATABASE_URL`, because it truncates tables.

The files, one by one

tests/test_basics.py

Geography, sources and download helpers.

- Metric distances agree with haversine; complexes chain neighbours; version picking keeps one version; the 16-day cycle arithmetic is right.

tests/test_anomaly.py

Pass segmentation, along-track coordinate, background.

- Includes the gradient test and a robustness test with 2 % wild outliers.

tests/test_events_stats.py

Events, Wilson intervals, control test, radial profile.

- Includes the 20-seed calibration test for the control test.

tests/test_pipeline.py

Reader, quality control, end-to-end pipeline, determinism, optional PostGIS round-trip.

- Checks a missing variable produces an error mentioning `inspect_file`, that synthetic and real cannot be mixed, and that two runs give identical results.

Check that it works

```
python -m pytest -q
```

Expected: `30 passed, 1 skipped` (the skipped one is the PostGIS test). With `TEST_DATABASE_URL` set to a throwaway database: `31 passed`.

Common errors

Symptom	Likely cause	Fix
Tests pass alone but fail together	Shared state between tests.	Use fixtures and temporary folders (`tmp_path`).

Symptom	Likely cause	Fix
A statistical test fails once in a while	A single random seed was used.	Test the property over many seeds and allow a small, justified number of exceptions.

Level 12: Notebook, README and publishing

Goal	Make the project understandable to a stranger, and put it on GitHub honestly.
Why now	A repository is read, not run. The notebook teaches the method, the README sets expectations, and the results section must say only what is true.

Concepts you need

- **The notebook** walks the pipeline stage by stage and was executed so outputs are embedded. It auto-detects real data and falls back to synthetic.
- **The README** opens with a status table of what is verified and what is not. Keep that honesty.
- **What to commit:** code, tests, `results/` from a real run, the reference CSVs, documentation. **Not:** `data/raw`, `.env`, `.venv`, `results_demo/`.

The files, one by one

notebooks/01_walkthrough.ipynb

Thirteen sections from “the question” to “store results in PostGIS”, with exercises.

- Each section has a short explanation, then runs the real project code.
- Re-run it after your real-data run and commit the executed version.

README.md

Status, quick start, real-data steps, method, limitations, licences, results placeholder.

- The Results section is a placeholder. Replace it only after a real run, with the date range, number of valid overpasses, the control-test numbers and two honest sentences.

Before you believe your first real result: the sanity checklist

1. **Inspect first.** Run `inspect_file`; confirm the variable names match `read_oco2.py`.
2. **Read the quality-control report.** If one rule removed almost everything, find out why.
3. **Compare the measured scatter with the reported uncertainty** (in `summary.json`). If the measured value is much larger, correlated errors dominate and thresholds may need raising.
4. **Look at the coverage map.** Which sources were never observed? Say so.
5. **Read the control test.** If the far flag rate is close to the near flag rate, the method is detecting noise. Do not report detections.
6. **Run the sensitivity analysis** (notebook section 10) on `exclude_km`. A result that appears at one radius only is fragile.
7. **Check outliers by eye.** Plot your three strongest events as example passes and ask whether they look like plumes or like artefacts.
8. **Write conclusions last,** and only what the numbers support.

Check that it works

```
git init && git add -A && git status    # confirm data/raw, .env and results_demo are NOT listed
```

Expected: Only code, tests, docs, reference data and (after a real run) `results/` are staged.

Common errors

Symptom	Likely cause	Fix
Large file rejected by GitHub	Raw data got committed.	Remove it from the index (`git rm -r --cached data/raw`) and check `.gitignore`.
Password committed	`.env` was tracked.	Rotate the password immediately; remove the file from history.

Appendix A: Troubleshooting index

Every error from the levels, in one place.

Symptom	Level	First thing to try
`ModuleNotFoundError: highveld_co2`	0	Run from the repository root
Points in the wrong place / absurd distances	1	`always_xy=True`; pass (lon, lat)
401 / 403 downloading	2	Authorise NASA GESDISC DATA ARCHIVE in your Earthdata profile
0 granules found	2	Check dates, product name, `OCO2_VERSION`
`KeyError: required variable`	3	`inspect_file`, then edit `REQUIRED`
All soundings removed by QC	3	Print `qf.value_counts()`
Refusing to mix synthetic and real	3	Separate folders
Every pass invalid	5	Print `n_far`, `extent_km`; relax thresholds
Hundreds of events	6	Raise `z_bin`; compare sigma with reported uncertainty
No events	6	Lower `min_enh_ppm`; check `z` distribution
Rate ratio NaN	7	No far flags: report near rate and a bound
Blank map	8	Print `grid.describe()`; lower `min_count`
`connection refused`	9	`docker compose ps`; check `DATABASE_URL`
`type "geometry" does not exist`	9	Use the PostGIS image
`can't adapt numpy.int64`	9	Use `_null_safe`
Large file rejected by GitHub	12	`git rm -r --cached data/raw`

Appendix B: Mistakes made while building this, and what they teach

These are real. Each was found by testing or by looking at output, not by thinking harder. Reading them is the fastest way to learn what to check in your own projects.

#	What went wrong	How it was found	Lesson
1	Attribution radius (30 km) was smaller than the background exclusion radius (40 km). 29 % of events fell in the gap and were left unattributed.	A test asserting most events were attributed failed; looking at the distance distribution showed a step at 40 km.	Parameters that describe one physical effect must be tied together, and the reason written next to them.
2	NaN values were written to the database as the value `'NaN'`, not NULL.	Anticipated from experience, then checked with `WHERE frequency = 'NaN'`.	Check what is really in the database, not what you meant to write.
3	The Fisher p-value printed exactly `0.0`.	Reading the notebook output.	A p-value of exactly zero is a numerical artefact; clamp and say so.

#	What went wrong	How it was found	Lesson
4	The SQL control test had 30 km hard-coded while Python used 40 km. The notebook claimed they matched; they did not.	Reading the notebook output carefully.	Duplicated logic drifts. Parameterise it, and add a regression test comparing both.
5	Used a pandas feature (`include_groups`) that needs 2.2 or newer.	Reviewing compatibility; replaced with a plain `groupby().sum()`.	Test on more than one library version before claiming compatibility.
6	The first synthetic data was too clean: almost no false alarms, so the control test showed a ratio of 383.	Noticing an implausibly perfect result.	A result that is too good is a warning. Simulate what makes the problem hard (correlated noise, artefacts).
7	A statistical test with a single random seed failed by chance.	A failing test whose cause was not a code bug.	First verify the statistic is calibrated (400 runs), then test over many seeds.
8	A value label on the control chart collided with its title.	Viewing the image.	Always look at the figure; no test would catch this.
9	A notebook example queried events near Kendal and returned nothing closer than 39.5 km.	Reading the output; the synthetic tracks only clip Kendal.	An example that returns empty or odd results teaches the wrong thing. Choose the example from the data.

Appendix C: Build order at a glance

Level	Files created	Command that proves it works
0	`requirements.txt`, `.gitignore`, `.env.example`, `config.py`, `conftest.py`, `__init__.py`	`python -c "from highveld_co2.config import DEFAULT_PARAMS"`
1	`geo.py`, `build_reference.py`, `extra_sources.csv`, `sources.py`	`python scripts/build_reference.py`
2	`download.py`, `inspect_file.py`	`python -m highveld_co2.inspect_file <file>`
3	`read_oco2.py`, `qc.py`	read one file and print the QC report
4	`synthetic.py`	`python -m highveld_co2.synthetic --days 730`
5	`anomaly.py`	`pytest tests/test_anomaly.py`
6	`events.py`	`pytest tests/test_events_stats.py -k plume`
7	`attribute.py`, `composite.py`	`pytest tests/test_events_stats.py`
8	`mapping.py`	`make demo`, then look at the figures
9	`schema.sql`, `db.py`, `docker-compose.yml`	`pipeline --db`; run twice
10	`pipeline.py`, `Makefile`	`make demo`
11	`tests/*.py`	`python -m pytest -q`
12	notebook, `README.md`	`git status`

Appendix D: Complete source code

Every file, exactly as in the repository. Use it to check your own typing, not to replace it.

highveld_co2/config.py

```
"""Central configuration: paths, study region and every tunable analysis parameter.

Everything that is a *choice* rather than a fact lives here, so that sensitivity
tests can change one number and re-run (see `Params` and `dataclasses.replace`).
"""
from dataclasses import dataclass
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
RAW_DIR = ROOT / "data" / "raw" / "oco2"          # real OCO-2 Lite files go here
SYNTH_DIR = ROOT / "data" / "raw" / "synthetic"   # synthetic test files go here
REF_DIR = ROOT / "data" / "reference"
PROC_DIR = ROOT / "data" / "processed"
RESULTS_DIR = ROOT / "results"

# Study region: South African Highveld (Mpumalanga coal-power cluster plus
# Johannesburg/Pretoria). West, South, East, North in degrees (EPSG:4326).
BBOX = (27.0, -28.0, 31.0, -24.5)

# Metric CRS used for every distance calculation. UTM zone 35S (EPSG:32735)
# covers 24E-30E; at the eastern edge of the box (31E) the scale error is
# well under 0.1 %, which is negligible next to the other uncertainties.
METRIC_CRS = "EPSG:32735"

# NASA GES DISC product: OCO-2 Level 2 bias-corrected XCO2 ("Lite"), daily files.
OCO2_SHORT_NAME = "OCO2_L2_Lite_FP"
OCO2_VERSION = None   # None = pick the newest version CMR returns; or e.g. "11.2r"


@dataclass(frozen=True)
class Params:
    """Analysis parameters. Frozen so a run can be reproduced from its Params."""

    # --- quality control ------------------------------------------------
    keep_modes: tuple = (0, 1)        # operation_mode: 0 nadir, 1 glint (2 target, 3 transition dropped)

    # --- segmenting soundings into overpasses ---------------------------
    pass_gap_s: float = 300.0         # time gap that starts a new pass (a pass over the box lasts ~1 min)
    min_soundings: int = 150          # good soundings needed for a usable pass
    min_far_soundings: int = 60       # far-from-source soundings needed to define a background
    min_extent_km: float = 100.0      # along-track length needed (short clips cannot define a background)

    # --- background and anomaly -----------------------------------------
    exclude_km: float = 40.0          # soundings closer than this to a source are not used for the background
    background_method: str = "running"   # "median" (one value per pass) or "running" (along-track median)
    running_window_km: float = 100.0
    sigma_floor_ppm: float = 0.3      # never trust a scatter estimate below this

    # --- binning and detection ------------------------------------------
    bin_km: float = 5.0               # along-track bin length
    min_bin_n: int = 5                # soundings needed in a bin
    min_enh_ppm: float = 1.0          # bin mean must exceed the background by this much...
    z_bin: float = 3.0                # ...and by this many standard errors

    # --- attribution and statistics -------------------------------------
    # attrib_km, near_km and cover_radius_km are deliberately EQUAL to exclude_km. The background
```

```
    # is built from soundings farther than exclude_km from any source, so a plume more than
    # exclude_km downwind is partly absorbed into the background and cannot be detected. Signal
    # therefore lives inside exclude_km; attributing or testing at a smaller radius would orphan
    # real detections (found the hard way: with 30 km, 29 % of synthetic events were unattributed).
    attrib_km: float = 40.0           # an event within this distance of a source is "attributed" to it
    near_km: float = 40.0             # "near" bins for the control test
    far_km: float = 60.0              # "far" (control) bins; the 40-60 km gap is a buffer
    cover_radius_km: float = 40.0     # a pass "observes" a complex if it has data within this radius
    complex_eps_km: float = 30.0      # sources closer than this (chained) form one complex
    hotspot_eps_km: float = 20.0      # events closer than this form one hotspot


DEFAULT_PARAMS = Params()
```

highveld_co2/geo.py

```
"""Small geography helpers: lon/lat <-> metres, great-circle distance."""
import numpy as np
from pyproj import Transformer

from .config import METRIC_CRS

_TO_M = Transformer.from_crs("EPSG:4326", METRIC_CRS, always_xy=True)
_TO_LL = Transformer.from_crs(METRIC_CRS, "EPSG:4326", always_xy=True)

EARTH_RADIUS_KM = 6371.0088


def to_metric(lon, lat):
    """lon/lat (degrees) -> x/y in metres in METRIC_CRS. Accepts scalars or arrays."""
    x, y = _TO_M.transform(np.asarray(lon, dtype=float), np.asarray(lat, dtype=float))
    return np.asarray(x), np.asarray(y)


def to_lonlat(x, y):
    """x/y metres in METRIC_CRS -> lon/lat degrees."""
    lon, lat = _TO_LL.transform(np.asarray(x, dtype=float), np.asarray(y, dtype=float))
    return np.asarray(lon), np.asarray(lat)


def haversine_km(lon1, lat1, lon2, lat2):
    """Great-circle distance in km. Used to cross-check the metric-CRS distances."""
    lon1, lat1, lon2, lat2 = map(np.radians, (lon1, lat1, lon2, lat2))
    a = np.sin((lat2 - lat1) / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin((lon2 - lon1) / 2) ** 2
    return 2 * EARTH_RADIUS_KM * np.arcsin(np.sqrt(a))
```

scripts/build_reference.py

```
"""Build data/reference/power_plants_highveld.csv from the WRI Global Power Plant Database.

    python scripts/build_reference.py

Source: World Resources Institute, Global Power Plant Database v1.3.0 (CC BY 4.0)
https://github.com/wri/global-power-plant-database
Only fossil plants >= 100 MW inside the study box (plus a 0.5 degree buffer) are kept.
"""
import io
import sys
from pathlib import Path
```

```
import pandas as pd
import requests

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from highveld_co2.config import BBOX, REF_DIR  # noqa: E402

URL = ("https://raw.githubusercontent.com/wri/global-power-plant-database/"
       "master/output_database/global_power_plant_database.csv")
FUELS = {"Coal", "Gas", "Oil"}
MIN_MW = 100
BUFFER = 0.5


def main():
    print("Downloading", URL)
    r = requests.get(URL, timeout=120)
    r.raise_for_status()
    df = pd.read_csv(io.StringIO(r.text), low_memory=False)

    w, s, e, n = BBOX
    keep = (
        (df.country == "ZAF")
        & df.primary_fuel.isin(FUELS)
        & (df.capacity_mw >= MIN_MW)
        & df.longitude.between(w - BUFFER, e + BUFFER)
        & df.latitude.between(s - BUFFER, n + BUFFER)
    )
    out = df.loc[keep, ["name", "primary_fuel", "capacity_mw", "latitude", "longitude",
                        "gppd_idnr", "geolocation_source"]].rename(columns={"primary_fuel": "fuel"})
    out = out.sort_values("capacity_mw", ascending=False)
    REF_DIR.mkdir(parents=True, exist_ok=True)
    path = REF_DIR / "power_plants_highveld.csv"
    out.to_csv(path, index=False)
    print(f"Wrote {len(out)} plants to {path}")
    print(out.to_string(index=False))


if __name__ == "__main__":
    main()
```

highveld_co2/sources.py

```
"""Known CO2 point sources: loading, grouping into complexes, fast nearest-source lookup."""
import geopandas as gpd
import numpy as np
import pandas as pd
from scipy.spatial import cKDTree
from sklearn.cluster import DBSCAN

from .config import DEFAULT_PARAMS, REF_DIR
from .geo import to_lonlat, to_metric


def load_sources(ref_dir=REF_DIR, min_capacity_mw: float = 0.0, include_extra: bool = True) -> gpd.GeoDataFrame:
    """Read the reference CSVs into one GeoDataFrame (EPSG:4326) with metric x/y columns.

    `extra_sources.csv` holds sources that are not power plants (e.g. Secunda). Their
    capacity is blank, so min_capacity_mw only filters rows that have a capacity.
    """
    frames = [pd.read_csv(ref_dir / "power_plants_highveld.csv")]
```

```
    extra = ref_dir / "extra_sources.csv"
    if include_extra and extra.exists():
        frames.append(pd.read_csv(extra))
    df = pd.concat(frames, ignore_index=True)
    df = df[df["capacity_mw"].isna() | (df["capacity_mw"] >= min_capacity_mw)].reset_index(drop=True)
    df.insert(0, "source_id", np.arange(len(df)))
    x, y = to_metric(df["longitude"].values, df["latitude"].values)
    df["x_m"], df["y_m"] = x, y
    return gpd.GeoDataFrame(df, geometry=gpd.points_from_xy(df.longitude, df.latitude), crs=4326)


def group_complexes(sources: gpd.GeoDataFrame, eps_km: float = DEFAULT_PARAMS.complex_eps_km):
    """Chain sources closer than eps_km into complexes.

    The satellite cannot separate sources a few km apart (a 5 km bin and a ~10 km wide swath),
    so attribution is only meaningful at complex level. DBSCAN with min_samples=1 is
    single-linkage: A-B within eps and B-C within eps puts A, B, C in one complex.

    Returns (sources with complex_id, complexes table). Complex ids are ordered by total
    capacity, largest first, so id 0 is the biggest complex.
    """
    xy = sources[["x_m", "y_m"]].values
    raw = DBSCAN(eps=eps_km * 1000.0, min_samples=1).fit_predict(xy)
    src = sources.copy()
    src["_raw"] = raw

    rows = []
    for raw_id, g in src.groupby("_raw"):
        cap = g["capacity_mw"].fillna(0).sum()
        lead = g.sort_values("capacity_mw", ascending=False, na_position="last").iloc[0]
        cx, cy = g["x_m"].mean(), g["y_m"].mean()
        lon, lat = to_lonlat(cx, cy)
        rows.append({"_raw": raw_id, "n_sources": len(g), "total_capacity_mw": cap,
                     "name": (lead["name"] + " complex") if len(g) > 1 else lead["name"],
                     "longitude": float(lon), "latitude": float(lat), "x_m": cx, "y_m": cy})
    comp = pd.DataFrame(rows).sort_values("total_capacity_mw", ascending=False).reset_index(drop=True)
    comp.insert(0, "complex_id", np.arange(len(comp)))
    mapping = dict(zip(comp["_raw"], comp["complex_id"]))
    src["complex_id"] = src["_raw"].map(mapping).astype(int)
    src = src.drop(columns="_raw")
    comp = comp.drop(columns="_raw")
    comp = gpd.GeoDataFrame(comp, geometry=gpd.points_from_xy(comp.longitude, comp.latitude), crs=4326)
    return src, comp


class SourceIndex:
    """Fast 'how far is the nearest source?' lookups in metres, via a KD-tree.

    Building the tree once and querying it for every sounding is what keeps the pipeline
    fast: a naive loop over sources for each of ~100k soundings would be far slower.
    """

    def __init__(self, sources: gpd.GeoDataFrame):
        self.sources = sources.reset_index(drop=True)
        self.tree = cKDTree(self.sources[["x_m", "y_m"]].values)

    def query(self, x_m, y_m):
        """Return (distance_km, source_row_index) of the nearest source for each point."""
        d, i = self.tree.query(np.c_[np.asarray(x_m), np.asarray(y_m)])
        return d / 1000.0, i

    def names(self, idx):
```

```
        return self.sources["name"].values[np.asarray(idx)]

    def complex_ids(self, idx):
        return self.sources["complex_id"].values[np.asarray(idx)]
```

highveld_co2/download.py

```
"""Download OCO-2 Lite files from NASA GES DISC with `earthaccess`.

One-time account setup (this is where most people get stuck):
  1. Free account at https://urs.earthdata.nasa.gov
  2. In your Earthdata profile -> Applications -> Authorized Apps, approve
     "NASA GESDISC DATA ARCHIVE". Without this, downloads fail with 401/403.
  3. Put EARTHDATA_USERNAME / EARTHDATA_PASSWORD in .env (see .env.example)

Why daily files matter: each Lite file covers the whole globe for one day, so a bounding box
cannot shrink the download. OCO-2 repeats its ground track every 16 days and sees the study
box only on ~2-3 days of every 16. `offsets` + `get --offsets` exploit that: download one
16-day window, learn which days cross the box, then fetch only those days in later cycles.

Commands
  python -m highveld_co2.download get --start 2023-06-01 --end 2023-08-31 [--max-files 100]
  python -m highveld_co2.download offsets --raw data/raw/oco2 --cycle-start 2023-06-01
  python -m highveld_co2.download get --start 2023-06-01 --end 2024-05-31 \\
         --cycle-start 2023-06-01 --offsets 2,7,11
"""
import argparse
import os
import re
from datetime import date, timedelta
from pathlib import Path

from dotenv import load_dotenv

from .config import BBOX, OCO2_SHORT_NAME, OCO2_VERSION, RAW_DIR


# ---------- pure helpers (unit-tested; no network) ---------------------------------

def parse_version(v: str) -> float:
    """'11.2r' -> 11.2 ; '10r' -> 10.0 ; unparsable -> -1."""
    m = re.match(r"\s*(\d+(?:\.\d+)?)", str(v))
    return float(m.group(1)) if m else -1.0


def pick_latest_version(granules, wanted=None):
    """Keep only granules of one product version.

    CMR can return the same day under several versions. Mixing them double-counts days and mixes
    retrieval algorithms, so we keep exactly one: `wanted`, or else the numerically newest.
    Returns (granules_kept, version_used).
    """
    def ver(g):
        return g["umm"]["CollectionReference"]["Version"]

    versions = sorted({ver(g) for g in granules}, key=parse_version)
    if not versions:
        return [], None
    use = wanted if wanted else versions[-1]
    return [g for g in granules if ver(g) == use], use
```

```
def cycle_days(cycle_start: date, offsets, start: date, end: date, period: int = 16):
    """All dates in [start, end] whose (date - cycle_start) mod `period` is in `offsets`."""
    offs = set(int(o) for o in offsets)
    out, d = [], start
    while d <= end:
        if (d - cycle_start).days % period in offs:
            out.append(d)
        d += timedelta(days=1)
    return out


def date_from_filename(name: str):
    """oco2_LtCO2_230715_B11014Ar_....nc4 -> date(2023, 7, 15); None if no match."""
    m = re.search(r"LtCO2_(\d{2})(\d{2})(\d{2})", name)
    if not m:
        return None
    yy, mm, dd = map(int, m.groups())
    return date(2000 + yy, mm, dd)


def overpass_offsets(raw_dir: Path, cycle_start: date, bbox=BBOX, period: int = 16):
    """Which day-offsets (0..15) of a 16-day cycle have soundings inside the box?

    Reads the files already downloaded for the first cycle. Returns a sorted list.
    """
    from .read_oco2 import read_lite_file

    found = set()
    for f in sorted(Path(raw_dir).glob("*.nc*")):
        d = date_from_filename(f.name)
        if d is None:
            continue
        off = (d - cycle_start).days
        if 0 <= off < period and len(read_lite_file(f, bbox)) > 0:
            found.add(off)
    return sorted(found)


# ---------- network part ----------------------------------------------------------

def search(start: str, end: str, bbox=BBOX, version=OCO2_VERSION):
    import earthaccess

    load_dotenv()
    earthaccess.login(strategy="environment")
    results = earthaccess.search_data(
        short_name=OCO2_SHORT_NAME, bounding_box=tuple(bbox), temporal=(start, end), count=2000)
    kept, used = pick_latest_version(results, version)
    print(f"CMR returned {len(results)} granules; keeping {len(kept)} of version {used}")
    return kept


def get(start, end, out_dir=RAW_DIR, max_files=200, cycle_start=None, offsets=None, version=OCO2_VERSION):
    import earthaccess

    granules = search(start, end, version=version)
    if cycle_start and offsets:
        wanted = set(cycle_days(date.fromisoformat(cycle_start), offsets,
                                date.fromisoformat(start), date.fromisoformat(end)))
        granules = [g for g in granules
                    if date_from_filename(g["umm"]["GranuleUR"]) in wanted]
```

```
        print(f"Cycle filter keeps {len(granules)} granules")
    if len(granules) > max_files:
        raise SystemExit(f"{len(granules)} files exceeds --max-files {max_files}. "
                         f"Narrow the dates, use --offsets, or raise --max-files.")
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    files = earthaccess.download(granules, str(out_dir))   # skips files already present
    print(f"{len(files)} files in {out_dir}")


def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd", required=True)
    g = sub.add_parser("get")
    g.add_argument("--start", required=True)
    g.add_argument("--end", required=True)
    g.add_argument("--out", default=str(RAW_DIR))
    g.add_argument("--max-files", type=int, default=200)
    g.add_argument("--cycle-start")
    g.add_argument("--offsets", type=lambda s: [int(x) for x in s.split(",")])
    o = sub.add_parser("offsets")
    o.add_argument("--raw", default=str(RAW_DIR))
    o.add_argument("--cycle-start", required=True)
    a = ap.parse_args()
    if a.cmd == "get":
        get(a.start, a.end, a.out, a.max_files, a.cycle_start, a.offsets)
    else:
        offs = overpass_offsets(Path(a.raw), date.fromisoformat(a.cycle_start))
        print("Day offsets with soundings in the box:", ",".join(map(str, offs)))
        print("Use:  --cycle-start", a.cycle_start, "--offsets", ",".join(map(str, offs)))


if __name__ == "__main__":
    main()
```

highveld_co2/inspect_file.py

```
"""Print the structure of a NetCDF file: groups, variables, shapes, units, fill values.

    python -m highveld_co2.inspect_file data/raw/oco2/oco2_LtCO2_230715_B11014Ar_*.nc4

Run this FIRST on a real file. If a variable the reader needs has a different name or
sits in a different group, this is how you find out.
"""
import sys

from netCDF4 import Dataset


def describe(path, max_vars_per_group=60):
    with Dataset(path) as ds:
        print(f"File: {path}")
        print("Global attributes:", {k: ds.getncattr(k) for k in list(ds.ncattrs())[:8]}, "...")

        def walk(node, prefix=""):
            vars_ = list(node.variables.items())
            print(f"\n[{prefix or '/'}]  {len(vars_)} variables")
            for name, v in vars_[:max_vars_per_group]:
                units = getattr(v, "units", "")
                print(f"  {name:38s} {str(v.dtype):9s} shape={v.shape}  {units}")
            if len(vars_) > max_vars_per_group:
```

```
                print(f"  ... {len(vars_) - max_vars_per_group} more")
            for gname, g in node.groups.items():
                walk(g, f"{prefix}/{gname}")

        walk(ds)


if __name__ == "__main__":
    if len(sys.argv) != 2:
        sys.exit(__doc__)
    describe(sys.argv[1])
```

highveld_co2/read_oco2.py

```
"""Read an OCO-2 Lite (L2 bias-corrected XCO2) NetCDF4 file into a pandas DataFrame.

Layout of a Lite file (each variable has one value per sounding):
  root:      xco2, xco2_uncertainty, xco2_quality_flag, latitude, longitude, time, sounding_id
  /Sounding: operation_mode, land_water_indicator, solar_zenith_angle, ...

If a variable name differs in your file version, run
    python -m highveld_co2.inspect_file path/to/file.nc4
and edit the names in REQUIRED / OPTIONAL below. That is the single place to fix.
"""
from pathlib import Path

import numpy as np
import pandas as pd
from netCDF4 import Dataset

from .config import BBOX

# canonical column  ->  path inside the file
REQUIRED = {
    "xco2": "xco2",
    "xco2_unc": "xco2_uncertainty",
    "qf": "xco2_quality_flag",
    "lat": "latitude",
    "lon": "longitude",
    "t_s": "time",                       # seconds since 1970-01-01 UTC
}
OPTIONAL = {
    "sounding_id": "sounding_id",
    "op_mode": "Sounding/operation_mode",         # 0 nadir, 1 glint, 2 target, 3 transition
    "land_water": "Sounding/land_water_indicator",  # 0 land, 1 water
    "sza": "solar_zenith_angle",
}


def _get(ds: Dataset, path: str):
    """Fetch one variable as a float array with NaN for fill values; None if absent."""
    node = ds
    *groups, var = path.split("/")
    for g in groups:
        if g not in node.groups:
            return None
        node = node.groups[g]
    if var not in node.variables:
        return None
    arr = node.variables[var][:]
    return np.ma.filled(np.ma.asarray(arr).astype("float64"), np.nan)
```

```
def read_lite_file(path, bbox=BBOX) -> pd.DataFrame:
    """Return the soundings inside `bbox` as a DataFrame (no quality filtering yet).

    Reading everything and masking afterwards is fine: a daily global file holds on the
    order of 10^5 soundings, so memory is not a concern.
    """
    path = Path(path)
    with Dataset(path) as ds:
        cols = {}
        for name, var in REQUIRED.items():
            a = _get(ds, var)
            if a is None:
                raise KeyError(f"{path.name}: required variable '{var}' not found. "
                               f"Run `python -m highveld_co2.inspect_file {path}` and update REQUIRED.")
            cols[name] = a
        for name, var in OPTIONAL.items():
            a = _get(ds, var)
            cols[name] = a if a is not None else np.full(len(cols["xco2"]), np.nan)
        synthetic = str(getattr(ds, "synthetic", "false")).lower() == "true"

    df = pd.DataFrame(cols)
    df["sounding_id"] = df["sounding_id"].round().astype("Int64")   # 16-digit ids fit exactly in float64
    w, s, e, n = bbox
    keep = df["lon"].between(w, e) & df["lat"].between(s, n) & df["t_s"].notna() & df["xco2"].notna()
    df = df.loc[keep].copy()
    df["time"] = pd.to_datetime(df["t_s"], unit="s", utc=True)
    df["file"] = path.name
    df.attrs["synthetic"] = synthetic
    return df.reset_index(drop=True)


def read_many(paths, bbox=BBOX) -> pd.DataFrame:
    """Read several files and concatenate. Files with no soundings in the box are skipped."""
    frames, synthetic_flags = [], []
    for p in sorted(paths):
        df = read_lite_file(p, bbox)
        synthetic_flags.append(df.attrs.get("synthetic", False))
        if len(df):
            frames.append(df)
    if not frames:
        raise ValueError("No soundings inside the study box in any file. Check BBOX and the file dates.")
    out = pd.concat(frames, ignore_index=True)
    # Mixing synthetic and real data would silently corrupt results, so refuse.
    if any(synthetic_flags) and not all(synthetic_flags):
        raise ValueError("Refusing to mix synthetic and real files in one run.")
    out.attrs["synthetic"] = bool(synthetic_flags and all(synthetic_flags))
    return out
```

highveld_co2/qc.py

```
"""Quality control: keep only soundings we are willing to interpret."""
import pandas as pd

from .config import DEFAULT_PARAMS, Params


def apply_qc(df: pd.DataFrame, params: Params = DEFAULT_PARAMS):
    """Filter soundings and report how many each rule removed.
```

```
    Rules (in order):
      1. xco2_quality_flag == 0   (NASA's own recommended 'good' flag)
      2. land only                (land_water_indicator == 0)
      3. operation_mode in params.keep_modes  (drop target / transition modes)
    A rule is skipped (and noted) when its column is entirely missing, so a file version
    without that variable still runs instead of silently dropping everything.

    Returns (clean DataFrame, report dict).
    """
    n0 = len(df)
    report = {"n_in": n0}
    ok = df["qf"] == 0
    report["removed_quality_flag"] = int((~ok).sum())

    if df["land_water"].notna().any():
        land = df["land_water"].fillna(0) == 0
        report["removed_water"] = int((ok & ~land).sum())
        ok &= land
    else:
        report["land_water_rule"] = "skipped (column missing)"

    if df["op_mode"].notna().any():
        mode_ok = df["op_mode"].isin(params.keep_modes) | df["op_mode"].isna()
        report["removed_operation_mode"] = int((ok & ~mode_ok).sum())
        ok &= mode_ok
    else:
        report["operation_mode_rule"] = "skipped (column missing)"

    out = df.loc[ok].reset_index(drop=True)
    out.attrs.update(df.attrs)
    report["n_out"] = len(out)
    report["fraction_kept"] = round(len(out) / n0, 4) if n0 else 0.0
    return out, report
```

highveld_co2/synthetic.py

```
"""Synthetic OCO-2-like files, so the whole pipeline can be built and tested without NASA access.

THESE ARE NOT OBSERVATIONS. Files carry the global attribute synthetic="true"; the reader
propagates it, the pipeline prints a banner and stamps every figure "SYNTHETIC DATA", and it
refuses to mix synthetic and real files.

What is simulated (so we know the right answer):
  * the real OCO-2 sampling pattern: a 16-day repeat, 3 tracks crossing the box, 8 footprints
    across a ~10 km swath, a sounding every ~2.25 km along track;
  * a background (trend + seasonal cycle + per-pass offset + along-track gradient + spatially
    correlated error + white noise matching the reported uncertainty);
  * cloud: patchy bad-quality soundings, and a few almost fully clouded passes;
  * CO2 plumes from the REAL power-plant locations, blown downwind with random wind;
  * two urban 'domes' (Johannesburg, Pretoria) that are NOT power plants but sit close to small
    ones, which makes them a deliberate attribution trap;
  * occasional localised retrieval artefacts that are not sources at all (false-alarm generators).
A truth table is written alongside the files so detection performance can be scored.

    python -m highveld_co2.synthetic --days 730 --out data/raw/synthetic
"""
import argparse
from datetime import date, datetime, timedelta, timezone
from pathlib import Path

import numpy as np
```

```
import pandas as pd
from netCDF4 import Dataset

from .config import SYNTH_DIR
from .geo import to_lonlat, to_metric
from .sources import load_sources

TRACKS = {1: 27.9, 6: 29.4, 12: 30.9}   # day offset in the 16-day cycle -> longitude at lat -26.25
HEADING = np.radians(352.0)             # sun-synchronous ground track runs ~8 degrees west of north
URBAN = [("Johannesburg", 28.04, -26.20, 1.5, 18.0),   # name, lon, lat, peak ppm, sigma km
         ("Pretoria", 28.19, -25.75, 1.2, 12.0)]
FRAME_STEP_KM = 2.25
N_FOOTPRINTS = 8
FOOTPRINT_SPACING_KM = 1.3


def _smooth_noise(rng, n, step_km, corr_km, scale):
    """Zero-mean noise with an exponential correlation length (km), rescaled to std=scale."""
    k = max(2, int(5 * corr_km / step_km))
    kernel = np.exp(-np.arange(k) * step_km / corr_km)
    white = rng.normal(size=n + 2 * k)
    sm = np.convolve(white, kernel, mode="full")[k:k + n]
    sm = (sm - sm.mean()) / (sm.std() + 1e-12)
    return sm * scale


def _plume_enhancement(x, y, src, wind_dir_deg, wind_ms, rng, amplitude_scale):
    """Sum of Gaussian-like plumes (ppm) at points (x, y) in metres; returns (enh, truth rows)."""
    enh = np.zeros_like(x)
    truth = []
    for r in src.itertuples():
        cap = 4000.0 if pd.isna(r.capacity_mw) else float(r.capacity_mw)
        operating = rng.uniform(0.5, 1.0)
        theta = np.radians(wind_dir_deg + rng.normal(0, 15))
        amp = min(4.0, 2.0 * (cap / 4000.0) ** 0.8 * operating * (4.0 / wind_ms) * amplitude_scale)
        dx, dy = x - r.x_m, y - r.y_m
        wx, wy = np.sin(theta), np.cos(theta)
        d = (dx * wx + dy * wy) / 1000.0        # km downwind
        c = (dx * wy - dy * wx) / 1000.0        # km crosswind
        sy = 2.5 + 0.12 * np.clip(d, 0, None)
        shape = np.where(d > 0, np.exp(-0.5 * (c / sy) ** 2) * np.exp(-d / 40.0) * (1 - np.exp(-d / 4.0)), 0.0)
        enh += amp * shape
        truth.append({"source": r.name, "amplitude_ppm": round(amp, 3),
                      "wind_dir_deg": round(float(np.degrees(theta)) % 360, 1), "wind_ms": round(wind_ms, 2)})
    return enh, truth


def _urban(x, y):
    out = np.zeros_like(x)
    for _, lon, lat, peak, sig in URBAN:
        ux, uy = to_metric(lon, lat)
        r2 = ((x - ux) ** 2 + (y - uy) ** 2) / 1000.0 ** 2
        out += peak * np.exp(-0.5 * r2 / sig ** 2)
    return out


def _simulate_pass(day: date, cycle_index: int, lon0: float, src, rng, amplitude_scale):
    """Return (DataFrame of every sounding on this pass, truth rows)."""
    n_frames = int(660 / FRAME_STEP_KM)
    u = (np.arange(n_frames) - n_frames / 2) * FRAME_STEP_KM          # km along track
    a = (np.arange(N_FOOTPRINTS) - (N_FOOTPRINTS - 1) / 2) * FOOTPRINT_SPACING_KM
    uu, aa = np.meshgrid(u, a, indexing="ij")
```

```
    fp = np.tile(np.arange(N_FOOTPRINTS), (n_frames, 1))
    frame = np.repeat(np.arange(n_frames)[:, None], N_FOOTPRINTS, axis=1)

    x0, y0 = to_metric(lon0, -26.25)
    d = np.array([np.sin(HEADING), np.cos(HEADING)])
    p = np.array([d[1], -d[0]])
    x = float(x0) + (uu * d[0] + aa * p[0]) * 1000.0
    y = float(y0) + (uu * d[1] + aa * p[1]) * 1000.0
    lon, lat = to_lonlat(x, y)

    # --- background ----------------------------------------------------------
    t_years = (day - date(2021, 1, 1)).days / 365.25
    base = 413.0 + 2.4 * t_years + 3.0 * np.cos(2 * np.pi * (day.timetuple().tm_yday - 100) / 365.25)
    pass_off = rng.normal(0, 0.4)
    gradient = rng.normal(0, 0.4) * (u / 200.0)
    corr = _smooth_noise(rng, n_frames, FRAME_STEP_KM, 25.0, 0.5)   # spatially correlated retrieval error
    fp_off = rng.normal(0, 0.08, N_FOOTPRINTS)
    unc = rng.uniform(0.45, 0.85, size=x.shape)
    white = rng.normal(size=x.shape) * unc * 1.1
    xco2 = base + pass_off + (gradient + corr)[:, None] + fp_off[None, :] + white

    # --- signal --------------------------------------------------------------
    wind_dir = rng.uniform(0, 360)
    wind_ms = float(np.clip(rng.lognormal(np.log(4.0), 0.4), 1.5, 10.0))
    plume, truth = _plume_enhancement(x, y, src, wind_dir, wind_ms, rng, amplitude_scale)
    xco2 = xco2 + plume + _urban(x, y)

    # --- retrieval artefacts: ~12 % of passes get one localised +/- bump that is NOT a source.
    # Real retrievals do this (bright/dark surfaces, thin cloud or aerosol); it is what makes
    # false alarms possible, and why the control test exists.
    if rng.random() < 0.12:
        uc = rng.uniform(-150, 150)
        amp = rng.uniform(1.2, 2.0) * rng.choice([-1, 1, 1])
        xco2 = xco2 + amp * np.exp(-0.5 * ((uu - uc) / 8.0) ** 2)

    # --- cloud / quality -----------------------------------------------------
    cloud_frac = 0.8 if rng.random() < 0.10 else rng.uniform(0.05, 0.55)
    cloud_field = _smooth_noise(rng, n_frames, FRAME_STEP_KM, 25.0, 1.0)
    bad_frame = cloud_field > np.quantile(cloud_field, 1 - cloud_frac)
    qf = (bad_frame[:, None] | (rng.random(x.shape) < 0.05)).astype(np.int16)

    t_start = datetime(day.year, day.month, day.day, 11, 30, tzinfo=timezone.utc).timestamp()
    t = t_start + frame * 0.333
    mode = np.full(x.shape, cycle_index % 2, dtype=np.int8)           # alternate nadir / glint by cycle
    ts = pd.to_datetime(t.ravel(), unit="s", utc=True).strftime("%Y%m%d%H%M%S").astype("int64")
    sid = ts.values * 100 + (frame.ravel() % 10) * 10 + fp.ravel() + 1

    df = pd.DataFrame({
        "latitude": lat.ravel(), "longitude": lon.ravel(), "time": t.ravel(),
        "xco2": xco2.ravel(), "xco2_uncertainty": unc.ravel(), "xco2_quality_flag": qf.ravel(),
        "sounding_id": sid, "operation_mode": mode.ravel(),
        "land_water_indicator": np.zeros(x.size, dtype=np.int8),
        "solar_zenith_angle": np.full(x.size, 40.0),
    })
    for tr in truth:
        tr["date"] = day.isoformat()
    return df, truth


def _write_nc(path: Path, df: pd.DataFrame):
    with Dataset(path, "w", format="NETCDF4") as ds:
        ds.title = "SYNTHETIC OCO-2-like Lite file for testing. NOT real observations."
```

```
        ds.synthetic = "true"
        ds.createDimension("sounding_id", len(df))
        g = ds.createGroup("Sounding")

        def put(node, name, arr, dtype, units=""):
            v = node.createVariable(name, dtype, ("sounding_id",), zlib=True)
            v[:] = arr
            if units:
                v.units = units

        put(ds, "latitude", df["latitude"].values, "f8", "degrees_north")
        put(ds, "longitude", df["longitude"].values, "f8", "degrees_east")
        put(ds, "time", df["time"].values, "f8", "seconds since 1970-01-01 00:00:00 UTC")
        put(ds, "xco2", df["xco2"].values, "f4", "ppm")
        put(ds, "xco2_uncertainty", df["xco2_uncertainty"].values, "f4", "ppm")
        put(ds, "xco2_quality_flag", df["xco2_quality_flag"].values, "i2")
        put(ds, "sounding_id", df["sounding_id"].values, "i8")
        put(ds, "solar_zenith_angle", df["solar_zenith_angle"].values, "f4", "degrees")
        put(g, "operation_mode", df["operation_mode"].values, "i1")
        put(g, "land_water_indicator", df["land_water_indicator"].values, "i1")


def make_files(out_dir=SYNTH_DIR, start="2021-06-01", n_days=730, seed=1, amplitude_scale=1.0):
    """Write one file per overpass day. Returns the list of paths."""
    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    src = load_sources()
    d0 = date.fromisoformat(start)
    paths, truth_all = [], []
    for i in range(n_days):
        day = d0 + timedelta(days=i)
        off = i % 16
        if off not in TRACKS:
            continue
        rng = np.random.default_rng(seed * 100003 + i)
        df, truth = _simulate_pass(day, i // 16, TRACKS[off], src, rng, amplitude_scale)
        path = out_dir / f"oco2_LtCO2_{day:%y%m%d}_B11014Ar_synthetic.nc4"
        _write_nc(path, df)
        paths.append(path)
        truth_all.extend(truth)
    pd.DataFrame(truth_all).to_csv(out_dir / "synthetic_truth.csv", index=False)
    return paths


def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--out", default=str(SYNTH_DIR))
    ap.add_argument("--start", default="2021-06-01")
    ap.add_argument("--days", type=int, default=730)
    ap.add_argument("--seed", type=int, default=1)
    ap.add_argument("--amplitude-scale", type=float, default=1.0)
    a = ap.parse_args()
    paths = make_files(a.out, a.start, a.days, a.seed, a.amplitude_scale)
    print(f"Wrote {len(paths)} synthetic overpass files to {a.out}  (NOT real data)")


if __name__ == "__main__":
    main()
```

highveld_co2/anomaly.py

```
"""From soundings to anomalies: segment overpasses, estimate a background, subtract it.

Why anomalies? Atmospheric CO2 is ~415 ppm everywhere and a power-plant plume adds only
~1-2 ppm. So the quantity of interest is not XCO2 itself but XCO2 minus what XCO2 *would* have
been without the local source. We estimate that 'what it would have been' (the background)
from soundings of the same overpass that are far from any known source.
"""
import numpy as np
import pandas as pd

from .config import DEFAULT_PARAMS, Params
from .geo import to_metric
from .sources import SourceIndex

MAD_TO_SIGMA = 1.4826   # for Gaussian data, sigma = 1.4826 * median absolute deviation


def add_metric(df: pd.DataFrame) -> pd.DataFrame:
    x, y = to_metric(df["lon"].values, df["lat"].values)
    df = df.copy()
    df["x_m"], df["y_m"] = x, y
    return df


def assign_passes(df: pd.DataFrame, gap_s: float = DEFAULT_PARAMS.pass_gap_s) -> pd.DataFrame:
    """Label each sounding with the overpass it belongs to.

    Soundings are sorted by time; a gap longer than `gap_s` starts a new pass. One pass over the
    box lasts about a minute and consecutive orbits are ~99 minutes apart, so any gap between
    ~2 minutes and ~90 minutes separates passes cleanly. pass_id = '<start date>_<n>'.
    """
    df = df.sort_values("t_s").reset_index(drop=True)
    new = np.r_[True, np.diff(df["t_s"].values) > gap_s]
    seg = np.cumsum(new) - 1
    start_date = pd.Series(df["time"].values[new]).dt.strftime("%Y-%m-%d").values   # one per segment
    # number segments that share a start date: 0, 1, ...
    k = pd.Series(start_date).groupby(start_date).cumcount().values
    ids = np.array([f"{d}_{n}" for d, n in zip(start_date, k)])
    df["pass_id"] = ids[seg]
    return df


def along_track_km(x_m: np.ndarray, y_m: np.ndarray, t_s: np.ndarray | None = None) -> np.ndarray:
    """Distance along the swath (km, starting at 0) for each sounding.

    The footprints of one pass lie on a nearly straight strip, so the first principal axis of
    their (x, y) coordinates *is* the along-track direction. Projecting each sounding onto it
    gives a 1-D coordinate. (Sorting by time instead would zigzag across the 8 footprints.)
    The axis is oriented so it increases with time, making results reproducible.
    """
    xy = np.c_[x_m - x_m.mean(), y_m - y_m.mean()]
    _, _, vt = np.linalg.svd(xy, full_matrices=False)
    s = xy @ vt[0] / 1000.0
    if t_s is not None and np.corrcoef(s, t_s)[0, 1] < 0:
        s = -s
    return s - s.min()


def estimate_background(s_km, xco2, far_mask, method: str, window_km: float, sigma_floor: float):
    """Background value at every sounding of one pass, plus a robust scatter.
```

```
    method="median":  one number, the median XCO2 of the far soundings.
    method="running": the median in consecutive `window_km` windows along the track, linearly
                      interpolated. This follows slow along-track trends (a real gradient or a
                      retrieval artefact) that a single number would leave in the anomaly.
    Scatter sigma = 1.4826 * MAD of the far soundings about the background (robust to the odd
    plume that leaks into the far field), floored at `sigma_floor`.
    """
    far_x = xco2[far_mask]
    bg_med = np.median(far_x)
    bg = np.full(len(xco2), bg_med)
    if method == "running":
        edges = np.arange(s_km.min(), s_km.max() + window_km, window_km)
        centers, meds = [], []
        for lo, hi in zip(edges[:-1], edges[1:]):
            m = far_mask & (s_km >= lo) & (s_km < hi)
            if m.sum() >= 10:
                centers.append(s_km[m].mean())
                meds.append(np.median(xco2[m]))
        if len(centers) >= 2:
            bg = np.interp(s_km, centers, meds)   # flat beyond the first/last centre
    sigma = max(sigma_floor, MAD_TO_SIGMA * np.median(np.abs(far_x - bg[far_mask])))
    return bg, float(sigma)


def compute_anomalies(df: pd.DataFrame, index: SourceIndex, params: Params = DEFAULT_PARAMS):
    """Add pass_id, s_km, dist_src_km, background_ppm, sigma_ppm, anomaly_ppm to each sounding.

    Returns (soundings of valid passes only, table with one row per pass incl. invalid ones).
    A pass is valid when it has enough good soundings, enough far-field soundings to define a
    background, and a long enough along-track extent.
    """
    df = assign_passes(add_metric(df), params.pass_gap_s)
    d_km, _ = index.query(df["x_m"].values, df["y_m"].values)
    df["dist_src_km"] = d_km
    df["s_km"] = np.nan
    df["background_ppm"] = np.nan
    df["sigma_ppm"] = np.nan

    rows, valid_ids = [], []
    for pid, g in df.groupby("pass_id", sort=False):
        s = along_track_km(g["x_m"].values, g["y_m"].values, g["t_s"].values)
        far = g["dist_src_km"].values >= params.exclude_km
        extent = float(s.max() - s.min())
        row = {"pass_id": pid, "obs_date": g["time"].iloc[0].date(), "t_start": g["time"].min(),
               "n_good": len(g), "n_far": int(far.sum()), "extent_km": extent,
               "unc_median_ppm": float(g["xco2_unc"].median()),
               "background_ppm": np.nan, "sigma_ppm": np.nan, "is_valid": False}
        if len(g) >= params.min_soundings and far.sum() >= params.min_far_soundings \
                and extent >= params.min_extent_km:
            bg, sigma = estimate_background(s, g["xco2"].values, far, params.background_method,
                                            params.running_window_km, params.sigma_floor_ppm)
            df.loc[g.index, "s_km"] = s
            df.loc[g.index, "background_ppm"] = bg
            df.loc[g.index, "sigma_ppm"] = sigma
            row.update(background_ppm=float(np.median(bg)), sigma_ppm=sigma, is_valid=True)
            valid_ids.append(pid)
        rows.append(row)

    passes = pd.DataFrame(rows)
    out = df[df["pass_id"].isin(valid_ids)].copy()
    out["anomaly_ppm"] = out["xco2"] - out["background_ppm"]
```

```
    out.attrs.update(df.attrs)
    return out.reset_index(drop=True), passes
```

highveld_co2/events.py

```
"""Detecting enhancement events: bin along track, flag significant bins, merge into events.

Single soundings are too noisy (~0.7 ppm) to flag one by one. Averaging the ~10-40 soundings in a
5 km along-track bin cuts the noise by sqrt(n), so a bin is flagged only when its mean anomaly
is both large in ppm (>= min_enh_ppm) and large relative to its standard error (>= z_bin).
Adjacent flagged bins in one pass are merged into one event.
"""
import numpy as np
import pandas as pd

from .config import DEFAULT_PARAMS, Params
from .sources import SourceIndex


def bin_passes(df: pd.DataFrame, index: SourceIndex, params: Params = DEFAULT_PARAMS) -> pd.DataFrame:
    """One row per (pass, along-track bin) with mean anomaly, standard error, z, distance to source."""
    d = df.copy()
    d["bin_idx"] = np.floor(d["s_km"] / params.bin_km).astype(int)
    g = d.groupby(["pass_id", "bin_idx"], sort=True).agg(
        n=("anomaly_ppm", "size"),
        mean_anomaly_ppm=("anomaly_ppm", "mean"),
        mean_xco2_ppm=("xco2", "mean"),
        sigma_ppm=("sigma_ppm", "first"),
        lat=("lat", "mean"), lon=("lon", "mean"),
        x_m=("x_m", "mean"), y_m=("y_m", "mean"),
        obs_date=("time", lambda t: t.iloc[0].date()),
    ).reset_index()
    g = g[g["n"] >= params.min_bin_n].copy()
    g["se_ppm"] = g["sigma_ppm"] / np.sqrt(g["n"])
    g["z"] = g["mean_anomaly_ppm"] / g["se_ppm"]
    dist, idx = index.query(g["x_m"].values, g["y_m"].values)
    g["dist_source_km"] = dist
    g["nearest_source_idx"] = idx
    g["flagged"] = (g["mean_anomaly_ppm"] >= params.min_enh_ppm) & (g["z"] >= params.z_bin)
    return g.reset_index(drop=True)


def group_events(bins: pd.DataFrame, params: Params = DEFAULT_PARAMS) -> pd.DataFrame:
    """Merge runs of adjacent flagged bins (same pass, consecutive bin_idx) into events."""
    f = bins[bins["flagged"]].sort_values(["pass_id", "bin_idx"]).copy()
    if f.empty:
        return pd.DataFrame(columns=[
            "event_id", "pass_id", "obs_date", "n_bins", "length_km", "n_soundings", "peak_anomaly_ppm",
            "mean_anomaly_ppm", "peak_z", "lat", "lon", "x_m", "y_m"])
    new_run = (f["pass_id"] != f["pass_id"].shift()) | (f["bin_idx"] - f["bin_idx"].shift() != 1)
    f["run"] = new_run.cumsum() - 1
    rows = []
    for run, g in f.groupby("run"):
        peak = g.loc[g["mean_anomaly_ppm"].idxmax()]
        w = g["mean_anomaly_ppm"].clip(lower=1e-6)       # weight positions by the size of the anomaly
        rows.append({
            "event_id": int(run), "pass_id": peak["pass_id"], "obs_date": peak["obs_date"],
            "n_bins": len(g), "length_km": len(g) * params.bin_km, "n_soundings": int(g["n"].sum()),
            "peak_anomaly_ppm": float(peak["mean_anomaly_ppm"]),
            "mean_anomaly_ppm": float(np.average(g["mean_anomaly_ppm"], weights=g["n"])),
            "peak_z": float(g["z"].max()),
```

```
            "lat": float(np.average(g["lat"], weights=w)), "lon": float(np.average(g["lon"], weights=w)),
            "x_m": float(np.average(g["x_m"], weights=w)), "y_m": float(np.average(g["y_m"], weights=w)),
        })
    return pd.DataFrame(rows)
```

highveld_co2/attribute.py

```
"""Linking events to sources, finding hotspots, and the statistics that keep us honest.

The key idea: because we hold ALL soundings (not just detections) we know how often each
source was actually *observed*, so we can report a detection frequency (events / observed passes)
instead of a bare count, and we can measure the false-alarm rate with control bins far from sources.
"""
import numpy as np
import pandas as pd
from scipy.spatial import cKDTree
from scipy.stats import fisher_exact
from sklearn.cluster import DBSCAN

from .config import DEFAULT_PARAMS, Params
from .geo import to_lonlat
from .sources import SourceIndex


def wilson_ci(k: int, n: int, z: float = 1.96):
    """95 % Wilson score interval for a proportion k/n. Sensible even for small n or k=0."""
    if n == 0:
        return (np.nan, np.nan)
    p = k / n
    denom = 1 + z ** 2 / n
    centre = (p + z ** 2 / (2 * n)) / denom
    half = z * np.sqrt(p * (1 - p) / n + z ** 2 / (4 * n ** 2)) / denom
    return (max(0.0, centre - half), min(1.0, centre + half))


def link_events(events: pd.DataFrame, index: SourceIndex, params: Params = DEFAULT_PARAMS) -> pd.DataFrame:
    """Attach nearest source, its distance, its complex, and whether it is within attrib_km."""
    ev = events.copy()
    if ev.empty:
        for c in ("nearest_source", "dist_source_km", "complex_id", "attributed"):
            ev[c] = pd.Series(dtype="object")
        return ev
    d, i = index.query(ev["x_m"].values, ev["y_m"].values)
    ev["nearest_source"] = index.names(i)
    ev["dist_source_km"] = d
    ev["complex_id"] = index.complex_ids(i)
    ev["attributed"] = d <= params.attrib_km
    return ev


def cluster_hotspots(events: pd.DataFrame, params: Params = DEFAULT_PARAMS) -> pd.DataFrame:
    """Group events into spatial hotspots and say whether each is explained by a known source.

    A hotspot with many events on different days and NO source nearby is the interesting case:
    here it is the Johannesburg/Pretoria urban area, which is not in the power-plant list.
    Returns the hotspots table; also adds 'hotspot_id' to `events` in place.
    """
    cols = ["hotspot_id", "n_events", "n_distinct_days", "lat", "lon", "mean_peak_ppm",
            "median_dist_source_km", "explained_by_source"]
    if events.empty:
        events["hotspot_id"] = pd.Series(dtype="int64")
```

```
        return pd.DataFrame(columns=cols)
    lab = DBSCAN(eps=params.hotspot_eps_km * 1000.0, min_samples=1).fit_predict(events[["x_m", "y_m"]].values)
    events["hotspot_id"] = lab
    rows = []
    for h, g in events.groupby("hotspot_id"):
        lon, lat = to_lonlat(g["x_m"].mean(), g["y_m"].mean())
        med = float(g["dist_source_km"].median())
        rows.append({"hotspot_id": int(h), "n_events": len(g), "n_distinct_days": g["obs_date"].nunique(),
                     "lat": float(lat), "lon": float(lon), "mean_peak_ppm": float(g["peak_anomaly_ppm"].mean()),
                     "median_dist_source_km": med, "explained_by_source": med <= params.attrib_km})
    return pd.DataFrame(rows).sort_values("n_distinct_days", ascending=False).reset_index(drop=True)


def complex_stats(bins: pd.DataFrame, sources, complexes, params: Params = DEFAULT_PARAMS) -> pd.DataFrame:
    """Per source complex: how many passes observed it, how many had an event, with a 95 % CI.

    'Observed' = the pass has >= 2 valid bins within cover_radius_km of any source in the complex.
    'Event'    = the pass has >= 1 flagged bin within that radius.
    """
    rows = []
    for _, c in complexes.iterrows():
        members = sources[sources["complex_id"] == c["complex_id"]]
        tree = cKDTree(members[["x_m", "y_m"]].values)
        d, _ = tree.query(bins[["x_m", "y_m"]].values)
        near = bins.loc[d / 1000.0 <= params.cover_radius_km]
        per_pass = near.groupby("pass_id").agg(n_bins=("flagged", "size"), n_flag=("flagged", "sum"))
        observed = int((per_pass["n_bins"] >= 2).sum())
        event_passes = int(((per_pass["n_bins"] >= 2) & (per_pass["n_flag"] >= 1)).sum())
        lo, hi = wilson_ci(event_passes, observed)
        rows.append({"complex_id": int(c["complex_id"]), "name": c["name"], "n_sources": int(c["n_sources"]),
                     "total_capacity_mw": float(c["total_capacity_mw"]),
                     "observed_passes": observed, "event_passes": event_passes,
                     "frequency": event_passes / observed if observed else np.nan,
                     "ci_low": lo, "ci_high": hi})
    return pd.DataFrame(rows)


def control_test(bins: pd.DataFrame, params: Params = DEFAULT_PARAMS, n_boot: int = 2000, seed: int = 0) -> dict:
    """Is the flagged-bin rate higher near sources than far from them?

    'Far' bins are the control: any flags there are false alarms (noise, retrieval artefacts,
    unlisted sources). If near and far flag equally often, our detections mean nothing.
    The CI comes from a bootstrap over PASSES, because bins in one pass are not independent
    (they share a background, a sigma and slowly varying errors), so a plain binomial CI would
    be too narrow.
    """
    near = bins["dist_source_km"] <= params.near_km
    far = bins["dist_source_km"] >= params.far_km
    nn, nf = int(near.sum()), int(bins.loc[near, "flagged"].sum())
    fn, ff = int(far.sum()), int(bins.loc[far, "flagged"].sum())
    out = {"near_bins": nn, "near_flagged": nf, "far_bins": fn, "far_flagged": ff,
           "near_rate": nf / nn if nn else np.nan, "far_rate": ff / fn if fn else np.nan}
    out["rate_ratio"] = out["near_rate"] / out["far_rate"] if ff > 0 and nn else np.nan
    if nn and fn:
        # One-sided Fisher exact test (is the near rate higher?). With thousands of bins the p-value can
        # underflow to exactly 0.0, which would read as "impossible"; clamp to the smallest honest bound.
        p = float(fisher_exact([[nf, nn - nf], [ff, fn - ff]], alternative="greater")[1])
        out["fisher_p"] = max(p, 1e-300)

    # per-pass counts: summing booleans counts the True values
    cnt = pd.DataFrame({"pass_id": bins["pass_id"], "nn": near, "nf": near & bins["flagged"],
                        "fn": far, "ff": far & bins["flagged"]}).groupby("pass_id").sum()
```

```
    rng = np.random.default_rng(seed)
    arr = cnt.values.astype(float)
    ratios = []
    for _ in range(n_boot):
        s = arr[rng.integers(0, len(arr), len(arr))].sum(axis=0)
        if s[0] > 0 and s[2] > 0 and s[3] > 0:
            ratios.append((s[1] / s[0]) / (s[3] / s[2]))
    if ratios:
        out["rate_ratio_ci"] = (float(np.percentile(ratios, 2.5)), float(np.percentile(ratios, 97.5)))
    return out
```

highveld_co2/composite.py

```
"""Compositing: average many sparse overpasses into a map and a distance profile.

One overpass shows almost nothing; hundreds averaged can reveal a persistent source.
"""
import numpy as np
import pandas as pd

from .config import BBOX


def grid_mean(bins: pd.DataFrame, res_deg: float = 0.1, min_count: int = 5, bbox=BBOX) -> pd.DataFrame:
    """Mean bin anomaly in res_deg cells (0.1 degrees ~ 10 km). Cells with < min_count bins are dropped."""
    w, s, e, n = bbox
    ix = np.floor((bins["lon"].values - w) / res_deg).astype(int)
    iy = np.floor((bins["lat"].values - s) / res_deg).astype(int)
    g = bins.assign(ix=ix, iy=iy).groupby(["ix", "iy"]).agg(
        mean_anomaly_ppm=("mean_anomaly_ppm", "mean"), n_bins=("mean_anomaly_ppm", "size"),
        n_passes=("pass_id", "nunique")).reset_index()
    g = g[g["n_bins"] >= min_count].copy()
    g["lon"] = w + (g["ix"] + 0.5) * res_deg
    g["lat"] = s + (g["iy"] + 0.5) * res_deg
    return g.reset_index(drop=True)


def radial_profile(bins: pd.DataFrame, edges_km=(0, 10, 20, 30, 40, 60, 80, 100), n_boot: int = 2000, seed: int =
0):
    """Mean anomaly vs distance to the nearest source, with a bootstrap-over-passes 95 % CI.

    Honest reading: the background was built from soundings > exclude_km from any source, so
    rings beyond that distance are ~0 BY CONSTRUCTION. The information is in the inner rings.
    """
    edges = np.asarray(edges_km, dtype=float)
    ring = np.digitize(bins["dist_source_km"].values, edges) - 1
    ok = (ring >= 0) & (ring < len(edges) - 1)
    b = bins.loc[ok].assign(ring=ring[ok])
    passes = b["pass_id"].unique()
    pidx = {p: i for i, p in enumerate(passes)}
    n_r = len(edges) - 1
    sums = np.zeros((len(passes), n_r))
    cnts = np.zeros((len(passes), n_r))
    for (p, r), g in b.groupby(["pass_id", "ring"]):
        sums[pidx[p], r] = g["mean_anomaly_ppm"].sum()
        cnts[pidx[p], r] = len(g)
    rng = np.random.default_rng(seed)
    boots = np.full((n_boot, n_r), np.nan)
    for k in range(n_boot):
        sel = rng.integers(0, len(passes), len(passes))
        s, c = sums[sel].sum(axis=0), cnts[sel].sum(axis=0)
```

```
        with np.errstate(invalid="ignore", divide="ignore"):
            boots[k] = np.where(c > 0, s / c, np.nan)
    tot_c = cnts.sum(axis=0)
    with np.errstate(invalid="ignore", divide="ignore"):
        mean = np.where(tot_c > 0, sums.sum(axis=0) / tot_c, np.nan)
    return pd.DataFrame({
        "r_lo_km": edges[:-1], "r_hi_km": edges[1:], "n_bins": tot_c.astype(int),
        "n_passes": [(cnts[:, r] > 0).sum() for r in range(n_r)],
        "mean_anomaly_ppm": mean,
        "ci_low": np.nanpercentile(boots, 2.5, axis=0), "ci_high": np.nanpercentile(boots, 97.5, axis=0)})
```

highveld_co2/mapping.py

```
"""Figures (matplotlib, saved as PNG) and an interactive folium map.

Design rules followed (from the data-visualisation guidance used for this project):
  * anomaly = polarity  -> diverging blue/red ramp with a NEUTRAL GREY midpoint (never a hue at 0);
  * one series          -> one colour, no legend box; several series -> legend AND a shape difference,
                           so identity never rests on colour alone;
  * thin marks, recessive grid, text in ink colours (never in the series colour);
  * synthetic data      -> every figure is stamped SYNTHETIC DATA.
"""
import folium
import matplotlib

matplotlib.use("Agg")
import matplotlib.patheffects as pe  # noqa: E402
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
import pandas as pd  # noqa: E402
from matplotlib.colors import LinearSegmentedColormap  # noqa: E402

from .attribute import wilson_ci  # noqa: E402
from .config import BBOX  # noqa: E402

SURFACE, INK, INK2, GRID = "#fcfcfb", "#0b0b0b", "#52514e", "#e4e3df"
BLUE, ORANGE, RED, NEUTRAL = "#2a78d6", "#eb6834", "#e34948", "#f0efec"
# diverging: three equal steps per arm, grey midpoint
DIV = LinearSegmentedColormap.from_list(
    "blue_grey_red", ["#184f95", "#2a78d6", "#9ec5f4", NEUTRAL, "#f3b0ad", "#e34948", "#a02a2a"])


def _style(ax):
    ax.set_facecolor(SURFACE)
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)
    for side in ("left", "bottom"):
        ax.spines[side].set_color(INK2)
        ax.spines[side].set_linewidth(0.8)
    ax.tick_params(colors=INK2, labelsize=9)
    ax.grid(True, color=GRID, linewidth=0.6)
    ax.set_axisbelow(True)
    ax.xaxis.label.set_color(INK2)
    ax.yaxis.label.set_color(INK2)


def _finish(fig, path, synthetic):
    fig.patch.set_facecolor(SURFACE)
    if synthetic:
        fig.text(0.5, 0.5, "SYNTHETIC DATA", rotation=28, fontsize=44, color=INK, alpha=0.10,
                 ha="center", va="center", weight="bold")
```

```
    fig.savefig(path, dpi=160, facecolor=SURFACE, bbox_inches="tight")
    plt.close(fig)
    return path


def fig_anomaly_map(grid, sources, complexes, path, bbox=BBOX, res_deg=0.1, vlim=1.5,
                    n_passes=None, synthetic=False):
    """Mean XCO2 anomaly per 0.1 degree cell, with known sources on top."""
    w, s, e, n = bbox
    nx, ny = int(round((e - w) / res_deg)), int(round((n - s) / res_deg))
    z = np.full((ny, nx), np.nan)
    g = grid[(grid["ix"] < nx) & (grid["iy"] < ny)]
    z[g["iy"].values, g["ix"].values] = g["mean_anomaly_ppm"].values

    fig, ax = plt.subplots(figsize=(7.4, 6.6))
    _style(ax)
    ax.grid(False)
    mesh = ax.pcolormesh(np.linspace(w, e, nx + 1), np.linspace(s, n, ny + 1), np.ma.masked_invalid(z),
                         cmap=DIV, vmin=-vlim, vmax=vlim, shading="flat")
    ax.scatter(sources["longitude"], sources["latitude"], marker="^", s=24 + sources["capacity_mw"].fillna(3000) /
70,
               c=INK, edgecolors=SURFACE, linewidths=1.2, zorder=5, label="Known source (size = capacity)")
    for _, c in complexes.sort_values("total_capacity_mw", ascending=False).head(4).iterrows():
        ax.annotate(c["name"].replace(" power station", "").replace(" complex", ""),
                    (c["longitude"], c["latitude"]), xytext=(6, 7), textcoords="offset points", fontsize=8.5,
                    color=INK, path_effects=[pe.withStroke(linewidth=2.6, foreground=SURFACE)], zorder=6)
    ax.set_xlim(w, e)
    ax.set_ylim(s, n)
    ax.set_aspect(1 / np.cos(np.radians((s + n) / 2)))
    ax.set_xlabel("Longitude (deg E)")
    ax.set_ylabel("Latitude (deg)")
    sub = f"{n_passes} valid overpasses; " if n_passes else ""
    ax.set_title(f"Mean XCO2 anomaly per 0.1 degree cell\n{sub}blank = fewer than 5 bins (not sampled)",
                 fontsize=11, color=INK, loc="left")
    cb = fig.colorbar(mesh, ax=ax, shrink=0.78, pad=0.02)
    cb.set_label("Anomaly vs pass background (ppm)", color=INK2)
    cb.ax.tick_params(colors=INK2, labelsize=9)
    ax.legend(loc="lower left", fontsize=8.5, frameon=True, facecolor=SURFACE, edgecolor=GRID, labelcolor=INK2)
    return _finish(fig, path, synthetic)


def fig_radial_profile(profile: pd.DataFrame, path, synthetic=False):
    """Mean anomaly vs distance to nearest source, with the bootstrap interval as a band."""
    p = profile.dropna(subset=["mean_anomaly_ppm"])
    x = (p["r_lo_km"] + p["r_hi_km"]) / 2
    fig, ax = plt.subplots(figsize=(7.0, 4.2))
    _style(ax)
    ax.axhline(0, color=INK2, linewidth=0.9)
    ax.fill_between(x, p["ci_low"], p["ci_high"], color=BLUE, alpha=0.18, linewidth=0)
    ax.plot(x, p["mean_anomaly_ppm"], color=BLUE, linewidth=2, marker="o", markersize=6,
            markeredgecolor=SURFACE, markeredgewidth=1.5)
    ax.set_xlabel("Distance to nearest known source (km)")
    ax.set_ylabel("Mean anomaly (ppm)")
    ax.set_title("XCO2 anomaly falls with distance from known sources\nband = 95 % interval, bootstrap over
overpasses",
                 fontsize=11, color=INK, loc="left")
    ax.text(0.99, 0.04, "Beyond the exclusion radius the background was built from these\nsoundings, so ~0 there
is by construction.",
            transform=ax.transAxes, ha="right", fontsize=8, color=INK2)
    return _finish(fig, path, synthetic)
```

```
def fig_example_pass(soundings, bins, pass_id, path, params, synthetic=False):
    """One overpass: sounding anomalies, 5 km bin means, flagged bins, and distance to nearest source."""
    s_ = soundings[soundings["pass_id"] == pass_id]
    b = bins[bins["pass_id"] == pass_id].copy()
    b["s_mid"] = (b["bin_idx"] + 0.5) * params.bin_km
    fig, (ax, ax2) = plt.subplots(2, 1, figsize=(8.0, 5.4), sharex=True, gridspec_kw={"height_ratios": [3, 1]})
    for a in (ax, ax2):
        _style(a)
    ax.axhline(0, color=INK2, linewidth=0.9)
    ax.scatter(s_["s_km"], s_["anomaly_ppm"], s=3, color=INK2, alpha=0.25, linewidths=0, label="Single soundings")
    ok = b[~b["flagged"]]
    fl = b[b["flagged"]]
    ax.plot(b["s_mid"], b["mean_anomaly_ppm"], color=BLUE, linewidth=1.2, alpha=0.6)
    ax.scatter(ok["s_mid"], ok["mean_anomaly_ppm"], s=22, color=BLUE, edgecolors=SURFACE, linewidths=1, zorder=4,
               label=f"{params.bin_km:g} km bin mean")
    ax.scatter(fl["s_mid"], fl["mean_anomaly_ppm"], s=60, marker="D", color=ORANGE, edgecolors=SURFACE,
               linewidths=1.2, zorder=5, label="Flagged bin (event)")
    ax.axhline(params.min_enh_ppm, color=INK2, linewidth=0.8, linestyle=(0, (4, 3)))
    ax.text(s_["s_km"].max(), params.min_enh_ppm, f" {params.min_enh_ppm:g} ppm threshold", fontsize=8,
color=INK2,
            va="bottom", ha="right")
    ax.set_ylabel("Anomaly (ppm)")
    ax.set_title(f"Overpass {pass_id}: anomaly along the track", fontsize=11, color=INK, loc="left")
    ax.legend(loc="upper left", fontsize=8.5, frameon=True, facecolor=SURFACE, edgecolor=GRID, labelcolor=INK2)
    ax2.plot(b["s_mid"], b["dist_source_km"], color=INK2, linewidth=1.4)
    ax2.axhline(params.attrib_km, color=INK2, linewidth=0.8, linestyle=(0, (4, 3)))
    ax2.set_ylabel("Dist. to\nsource (km)")
    ax2.set_xlabel("Distance along track (km)")
    ax2.set_ylim(bottom=0)
    return _finish(fig, path, synthetic)


def fig_control(control: dict, path, synthetic=False):
    """Flagged-bin rate near sources vs far from them (the control test), with 95 % Wilson intervals."""
    vals = [("Near sources", control["near_flagged"], control["near_bins"]),
            ("Far from sources\n(control)", control["far_flagged"], control["far_bins"])]
    fig, ax = plt.subplots(figsize=(5.4, 4.0))
    _style(ax)
    for i, (lab, k, n) in enumerate(vals):
        r = k / n if n else 0
        lo, hi = wilson_ci(k, n)
        ax.bar(i, r * 100, width=0.5, color=BLUE)
        ax.errorbar(i, r * 100, yerr=[[ (r - lo) * 100], [(hi - r) * 100]], color=INK2, capsize=4, linewidth=1.2)
        ax.text(i, hi * 100 + 0.4, f"{r * 100:.1f} %\n({k}/{n})", ha="center", fontsize=9, color=INK)
    ax.set_xticks(range(2))
    ax.set_xticklabels([v[0] for v in vals])
    ax.set_ylabel("Bins flagged (%)")
    top = max(wilson_ci(k, n)[1] * 100 for _, k, n in vals if n) if any(n for _, _, n in vals) else 1.0
    ax.set_ylim(0, top * 1.35)          # headroom so the value labels never hit the title
    ax.set_title("Do detections concentrate near sources?\nerror bars = 95 % Wilson interval", fontsize=11,
                 color=INK, loc="left", pad=12)
    return _finish(fig, path, synthetic)


def interactive_map(sources, complexes, events, hotspots, path, bbox=BBOX):
    """Folium map: sources, events (by peak anomaly), hotspots. Open the HTML in any browser."""
    w, s, e, n = bbox
    m = folium.Map(location=[(s + n) / 2, (w + e) / 2], zoom_start=7, tiles="OpenStreetMap")
    folium.Rectangle([[s, w], [n, e]], color="#52514e", weight=1, fill=False).add_to(m)

    fs = folium.FeatureGroup(name="Known sources")
    for _, r in sources.iterrows():
```

```
        cap = "n/a" if pd.isna(r["capacity_mw"]) else f"{r['capacity_mw']:,.0f} MW"
        folium.CircleMarker([r["latitude"], r["longitude"]], radius=4 + (0 if pd.isna(r["capacity_mw"]) else
r["capacity_mw"] / 800),
                            color="#0b0b0b", weight=1, fill=True, fill_color="#52514e", fill_opacity=0.8,
                            tooltip=f"{r['name']} ({cap})").add_to(fs)
    fs.add_to(m)

    fe = folium.FeatureGroup(name="Events")
    for _, r in events.iterrows():
        folium.CircleMarker([r["lat"], r["lon"]], radius=3 + 2 * r["peak_anomaly_ppm"], color="#eb6834", weight=1,
                            fill=True, fill_color="#eb6834", fill_opacity=0.6,
                            tooltip=f"{r['obs_date']} | peak {r['peak_anomaly_ppm']:.2f} ppm | "
                                    f"{r['dist_source_km']:.0f} km from {r['nearest_source']}").add_to(fe)
    fe.add_to(m)

    fh = folium.FeatureGroup(name="Hotspots (>= 3 distinct days)")
    for _, r in hotspots[hotspots["n_distinct_days"] >= 3].iterrows():
        folium.Circle([r["lat"], r["lon"]], radius=20000, color="#e34948", weight=2, fill=False,
                      tooltip=f"hotspot {r['hotspot_id']}: {r['n_events']} events on {r['n_distinct_days']} days;
"
                              f"explained by source: {r['explained_by_source']}").add_to(fh)
    fh.add_to(m)
    folium.LayerControl().add_to(m)
    m.save(str(path))
    return path
```

sql/schema.sql

```
-- PostGIS schema for the Highveld CO2 project.
-- Tables are listed parents-first so foreign keys resolve. All geometries are WGS84 points.
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE IF NOT EXISTS source_complexes (
    complex_id        INTEGER PRIMARY KEY,
    name              TEXT NOT NULL,
    n_sources         INTEGER NOT NULL,
    total_capacity_mw DOUBLE PRECISION,
    geom              GEOMETRY(Point, 4326) NOT NULL
);

CREATE TABLE IF NOT EXISTS sources (
    source_id          INTEGER PRIMARY KEY,
    name               TEXT NOT NULL,
    fuel               TEXT,
    capacity_mw        DOUBLE PRECISION,
    complex_id         INTEGER NOT NULL REFERENCES source_complexes(complex_id),
    geolocation_source TEXT,
    geom               GEOMETRY(Point, 4326) NOT NULL
);

-- One row per OCO-2 overpass of the study box (valid or not)
CREATE TABLE IF NOT EXISTS passes (
    pass_id        TEXT PRIMARY KEY,
    obs_date       DATE NOT NULL,
    t_start        TIMESTAMPTZ,
    n_good         INTEGER,
    n_far          INTEGER,
    extent_km      DOUBLE PRECISION,
    background_ppm DOUBLE PRECISION,
    sigma_ppm      DOUBLE PRECISION,
    unc_median_ppm DOUBLE PRECISION,
```

```
    is_valid       BOOLEAN NOT NULL
);

-- One row per 5 km along-track bin of a valid pass
CREATE TABLE IF NOT EXISTS bins (
    pass_id           TEXT NOT NULL REFERENCES passes(pass_id),
    bin_idx           INTEGER NOT NULL,
    n                 INTEGER NOT NULL,
    mean_anomaly_ppm  DOUBLE PRECISION,
    se_ppm            DOUBLE PRECISION,
    z                 DOUBLE PRECISION,
    dist_source_km    DOUBLE PRECISION,
    flagged           BOOLEAN NOT NULL,
    geom              GEOMETRY(Point, 4326) NOT NULL,
    PRIMARY KEY (pass_id, bin_idx)
);

-- One row per enhancement event (merged run of flagged bins)
CREATE TABLE IF NOT EXISTS events (
    event_id          INTEGER PRIMARY KEY,
    pass_id           TEXT NOT NULL REFERENCES passes(pass_id),
    obs_date          DATE NOT NULL,
    n_bins            INTEGER,
    length_km         DOUBLE PRECISION,
    peak_anomaly_ppm  DOUBLE PRECISION,
    mean_anomaly_ppm  DOUBLE PRECISION,
    peak_z            DOUBLE PRECISION,
    nearest_source    TEXT,
    dist_source_km    DOUBLE PRECISION,
    attributed        BOOLEAN,
    complex_id        INTEGER REFERENCES source_complexes(complex_id),
    hotspot_id        INTEGER,
    geom              GEOMETRY(Point, 4326) NOT NULL
);

CREATE TABLE IF NOT EXISTS complex_stats (
    complex_id      INTEGER PRIMARY KEY REFERENCES source_complexes(complex_id),
    observed_passes INTEGER,
    event_passes    INTEGER,
    frequency       DOUBLE PRECISION,
    ci_low          DOUBLE PRECISION,
    ci_high         DOUBLE PRECISION
);

CREATE INDEX IF NOT EXISTS bins_geom_gix    ON bins    USING GIST (geom);
CREATE INDEX IF NOT EXISTS events_geom_gix  ON events  USING GIST (geom);
CREATE INDEX IF NOT EXISTS sources_geom_gix ON sources USING GIST (geom);
CREATE INDEX IF NOT EXISTS events_date_ix   ON events (obs_date);
CREATE INDEX IF NOT EXISTS bins_flag_ix     ON bins (flagged) WHERE flagged;
```

highveld_co2/db.py

```
"""PostgreSQL/PostGIS persistence and example spatial queries.

    docker compose up -d            # starts PostGIS (see docker-compose.yml)
    python -m highveld_co2.pipeline --raw data/raw/oco2 --out results --db
"""
import os

import geopandas as gpd
import pandas as pd
```

```
from dotenv import load_dotenv
from sqlalchemy import create_engine, text

from .config import DEFAULT_PARAMS, ROOT

SCHEMA_SQL = (ROOT / "sql" / "schema.sql").read_text()
DEFAULT_URL = "postgresql+psycopg2://co2:co2@localhost:5432/co2"


def get_engine():
    load_dotenv(ROOT / ".env")
    return create_engine(os.getenv("DATABASE_URL", DEFAULT_URL))


def _null_safe(df: pd.DataFrame) -> pd.DataFrame:
    """NaN -> None for every non-geometry column.

    Without this, NaN floats are written as the value 'NaN' (not NULL), which silently breaks
    AVG(), COUNT(col) and any 'IS NULL' test. astype(object) also turns numpy ints/bools into
    plain Python values, which the database driver can adapt.
    """
    out = df.copy()
    for c in out.columns:
        if c == "geom":
            continue
        out[c] = out[c].astype(object).where(out[c].notna(), None)
    return out


def _points(df, lon="lon", lat="lat", cols=()):
    g = gpd.GeoDataFrame(df[list(cols)].copy(), geometry=gpd.points_from_xy(df[lon], df[lat]), crs=4326)
    return _null_safe(g.rename_geometry("geom"))


def save_all(sources, complexes, passes, bins, events, cstats, engine=None):
    """Create the schema if needed, replace all table contents, then load. Idempotent."""
    engine = engine or get_engine()
    with engine.begin() as con:
        con.execute(text(SCHEMA_SQL))
        con.execute(text("TRUNCATE complex_stats, events, bins, passes, sources, source_complexes CASCADE"))

    _points(complexes, "longitude", "latitude",
            ["complex_id", "name", "n_sources", "total_capacity_mw"]).to_postgis(
        "source_complexes", engine, if_exists="append", index=False)
    _points(sources, "longitude", "latitude",
            ["source_id", "name", "fuel", "capacity_mw", "complex_id", "geolocation_source"]).to_postgis(
        "sources", engine, if_exists="append", index=False)

    pcols = ["pass_id", "obs_date", "t_start", "n_good", "n_far", "extent_km", "background_ppm",
             "sigma_ppm", "unc_median_ppm", "is_valid"]
    _null_safe(passes[pcols]).to_sql("passes", engine, if_exists="append", index=False)

    _points(bins, cols=["pass_id", "bin_idx", "n", "mean_anomaly_ppm", "se_ppm", "z", "dist_source_km",
                        "flagged"]).to_postgis("bins", engine, if_exists="append", index=False, chunksize=5000)

    if len(events):
        ecols = ["event_id", "pass_id", "obs_date", "n_bins", "length_km", "peak_anomaly_ppm",
                 "mean_anomaly_ppm", "peak_z", "nearest_source", "dist_source_km", "attributed",
                 "complex_id", "hotspot_id"]
        _points(events, cols=ecols).to_postgis("events", engine, if_exists="append", index=False)

    scols = ["complex_id", "observed_passes", "event_passes", "frequency", "ci_low", "ci_high"]
```

```
    _null_safe(cstats[scols]).to_sql("complex_stats", engine, if_exists="append", index=False)
    return {t: int(pd.read_sql(text(f"SELECT COUNT(*) FROM {t}"), engine).iloc[0, 0])
            for t in ("source_complexes", "sources", "passes", "bins", "events", "complex_stats")}


EXAMPLE_QUERIES = {
    # detection frequency per complex, best-sampled first
    "complex_frequency": """
        SELECT c.name, s.observed_passes, s.event_passes,
               ROUND(s.frequency::numeric, 3) AS frequency,
               ROUND(s.ci_low::numeric, 3) AS ci_low, ROUND(s.ci_high::numeric, 3) AS ci_high
        FROM complex_stats s JOIN source_complexes c USING (complex_id)
        ORDER BY s.observed_passes DESC;""",
    # events within :km of a point (geography => true metres on the ellipsoid)
    "events_near_point": """
        SELECT event_id, obs_date, ROUND(peak_anomaly_ppm::numeric, 2) AS peak_ppm,
               ROUND((ST_Distance(geom::geography,
                      ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::geography) / 1000)::numeric, 1) AS km
        FROM events
        WHERE ST_DWithin(geom::geography, ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::geography, :km * 1000)
        ORDER BY km;""",
    # events that no known source explains
    "unattributed_events": """
        SELECT event_id, obs_date, ROUND(peak_anomaly_ppm::numeric, 2) AS peak_ppm,
               nearest_source, ROUND(dist_source_km::numeric, 1) AS dist_km
        FROM events WHERE NOT attributed ORDER BY peak_anomaly_ppm DESC LIMIT 20;""",
    # seasonality of events
    "events_by_month": """
        SELECT date_trunc('month', obs_date)::date AS month, COUNT(*) AS events,
               ROUND(AVG(peak_anomaly_ppm)::numeric, 2) AS mean_peak_ppm
        FROM events GROUP BY 1 ORDER BY 1;""",
    # the control test in SQL: flag rate near vs far. :near and :far must equal Params.near_km / far_km
    # (defaults below), otherwise it will not reproduce the Python numbers.
    "control_test": """
        SELECT CASE WHEN dist_source_km <= :near THEN 'near' WHEN dist_source_km >= :far THEN 'far' END AS zone,
               COUNT(*) AS bins, SUM(flagged::int) AS flagged,
               ROUND(AVG(flagged::int)::numeric, 4) AS flag_rate
        FROM bins WHERE dist_source_km <= :near OR dist_source_km >= :far GROUP BY 1 ORDER BY 1;""",
}


# bind-parameter defaults, taken from the analysis defaults so SQL and Python agree out of the box
QUERY_DEFAULTS = {"control_test": {"near": DEFAULT_PARAMS.near_km, "far": DEFAULT_PARAMS.far_km}}


def run_query(name, engine=None, **params):
    """Run one of EXAMPLE_QUERIES. Extra keyword arguments are SQL bind parameters (:lon, :lat, :km ...)."""
    engine = engine or get_engine()
    with engine.connect() as con:
        return pd.read_sql(text(EXAMPLE_QUERIES[name]), con, params={**QUERY_DEFAULTS.get(name, {}), **params})
```

highveld_co2/pipeline.py

```
"""Run the whole pipeline: files -> QC -> anomalies -> events -> attribution -> figures -> (PostGIS).

    python -m highveld_co2.pipeline --raw data/raw/oco2 --out results            # real data
    python -m highveld_co2.pipeline --raw data/raw/synthetic --out results_demo  # synthetic demo
    add --db to load PostGIS (needs docker compose up -d)
"""
import argparse
import dataclasses
```

```
import json
from pathlib import Path

import numpy as np
import pandas as pd

from . import mapping
from .anomaly import compute_anomalies
from .attribute import cluster_hotspots, complex_stats, control_test, link_events
from .composite import grid_mean, radial_profile
from .config import DEFAULT_PARAMS, RAW_DIR, RESULTS_DIR, Params
from .events import bin_passes, group_events
from .qc import apply_qc
from .read_oco2 import read_many
from .sources import SourceIndex, group_complexes, load_sources


def _json_default(o):
    if isinstance(o, (np.integer,)):
        return int(o)
    if isinstance(o, (np.floating,)):
        return None if np.isnan(o) else float(o)
    if isinstance(o, (np.bool_,)):
        return bool(o)
    if isinstance(o, (pd.Timestamp,)):
        return o.isoformat()
    return str(o)


def run(raw_dir=RAW_DIR, out_dir=RESULTS_DIR, params: Params = DEFAULT_PARAMS, use_db=False,
        make_figures=True, verbose=True) -> dict:
    """Execute every stage and return all intermediate tables (the notebook uses this too)."""
    log = print if verbose else (lambda *a, **k: None)
    raw_dir, out_dir = Path(raw_dir), Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

    files = sorted(list(raw_dir.glob("*.nc4")) + list(raw_dir.glob("*.nc")))
    if not files:
        raise SystemExit(f"No .nc/.nc4 files in {raw_dir}. See README step 3 (download) or use the synthetic
demo.")
    log(f"[1/8] reading {len(files)} files from {raw_dir}")
    df = read_many(files)
    synthetic = bool(df.attrs.get("synthetic"))
    if synthetic:
        log("      *** SYNTHETIC DATA: results below are a software test, NOT observations ***")

    log("[2/8] quality control")
    clean, qc_report = apply_qc(df, params)
    log(f"      kept {qc_report['n_out']:,} of {qc_report['n_in']:,} soundings
({qc_report['fraction_kept']:.0%})")

    log("[3/8] sources")
    sources, complexes = group_complexes(load_sources(), params.complex_eps_km)
    index = SourceIndex(sources)
    log(f"      {len(sources)} sources in {len(complexes)} complexes")

    log("[4/8] overpasses, background, anomalies")
    soundings, passes = compute_anomalies(clean, index, params)
    n_valid = int(passes["is_valid"].sum())
    log(f"      {len(passes)} overpasses, {n_valid} valid")
    if n_valid == 0:
        raise SystemExit("No valid overpasses. Loosen min_soundings / min_extent_km in config.Params.")
```

```
    log("[5/8] binning and event detection")
    bins = bin_passes(soundings, index, params)
    events = link_events(group_events(bins, params), index, params)
    hotspots = cluster_hotspots(events, params)
    log(f"      {len(bins):,} bins, {int(bins['flagged'].sum())} flagged, {len(events)} events, {len(hotspots)}
hotspots")

    log("[6/8] statistics")
    cstats = complex_stats(bins, sources, complexes, params)
    control = control_test(bins, params)
    profile = radial_profile(bins)
    grid = grid_mean(bins)
    log(f"      flag rate near sources {control['near_rate']:.1%} vs far {control['far_rate']:.1%}; "
        f"ratio {control.get('rate_ratio', float('nan')):.1f}")

    log("[7/8] writing tables" + (" and figures" if make_figures else ""))
    passes.to_csv(out_dir / "passes.csv", index=False)
    bins.drop(columns=["x_m", "y_m", "nearest_source_idx"]).to_csv(out_dir / "bins.csv.gz", index=False)
    events.drop(columns=["x_m", "y_m"]).to_csv(out_dir / "events.csv", index=False)
    hotspots.to_csv(out_dir / "hotspots.csv", index=False)
    cstats.to_csv(out_dir / "complex_stats.csv", index=False)
    profile.to_csv(out_dir / "radial_profile.csv", index=False)

    if make_figures:
        mapping.fig_anomaly_map(grid, sources, complexes, out_dir / "fig_anomaly_map.png",
                                n_passes=n_valid, synthetic=synthetic)
        mapping.fig_radial_profile(profile, out_dir / "fig_radial_profile.png", synthetic=synthetic)
        mapping.fig_control(control, out_dir / "fig_control_test.png", synthetic=synthetic)
        if len(events):
            pid = events.sort_values("peak_anomaly_ppm", ascending=False)["pass_id"].iloc[0]
        else:
            pid = passes.loc[passes["is_valid"], "pass_id"].iloc[0]
        mapping.fig_example_pass(soundings, bins, pid, out_dir / "fig_example_pass.png", params,
synthetic=synthetic)
        mapping.interactive_map(sources, complexes, events, hotspots, out_dir / "map.html")

    summary = {
        "synthetic_data": synthetic,
        "n_files": len(files),
        "date_range": [str(df["time"].min()), str(df["time"].max())],
        "qc": qc_report,
        "n_overpasses": len(passes), "n_valid_overpasses": n_valid,
        "n_bins": len(bins), "n_flagged_bins": int(bins["flagged"].sum()),
        "n_events": len(events), "n_hotspots": len(hotspots),
        "noise": {"median_sigma_ppm": float(passes.loc[passes.is_valid, "sigma_ppm"].median()),
                  "median_reported_uncertainty_ppm": float(passes.loc[passes.is_valid,
"unc_median_ppm"].median())},
        "control_test": control,
        "params": dataclasses.asdict(params),
    }
    (out_dir / "summary.json").write_text(json.dumps(summary, indent=2, default=_json_default))

    if use_db:
        log("[8/8] loading PostGIS")
        from .db import save_all
        counts = save_all(sources, complexes, passes, bins, events, cstats)
        log(f"      rows: {counts}")
    else:
        log("[8/8] database step skipped (use --db)")
    log(f"done -> {out_dir}")
    return dict(df=df, clean=clean, soundings=soundings, passes=passes, bins=bins, events=events,
```

```
                hotspots=hotspots, sources=sources, complexes=complexes, cstats=cstats, control=control,
                profile=profile, grid=grid, summary=summary, qc=qc_report, index=index, params=params)


def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--raw", default=str(RAW_DIR))
    ap.add_argument("--out", default=str(RESULTS_DIR))
    ap.add_argument("--db", action="store_true")
    ap.add_argument("--exclude-km", type=float)
    ap.add_argument("--min-enh", type=float)
    ap.add_argument("--z-bin", type=float)
    a = ap.parse_args()
    over = {k: v for k, v in {"exclude_km": a.exclude_km, "min_enh_ppm": a.min_enh, "z_bin": a.z_bin}.items()
            if v is not None}
    run(a.raw, a.out, dataclasses.replace(DEFAULT_PARAMS, **over), use_db=a.db)


if __name__ == "__main__":
    main()
```

Makefile

```
.PHONY: setup test demo real db-up db-down notebook clean

PY ?= python

setup:            ## install dependencies
    $(PY) -m pip install -r requirements.txt

test:             ## run the unit + integration tests (30 pass; the PostGIS test needs TEST_DATABASE_URL)
    $(PY) -m pytest -q

demo:             ## full pipeline on SYNTHETIC data (a software test, not a result)
    $(PY) -m highveld_co2.synthetic --days 730
    $(PY) -m highveld_co2.pipeline --raw data/raw/synthetic --out results_demo

real:             ## full pipeline on REAL OCO-2 files in data/raw/oco2 -> results/
    $(PY) -m highveld_co2.pipeline --raw data/raw/oco2 --out results

db-up:            ## start PostGIS in Docker
    docker compose up -d

db-down:
    docker compose down

notebook:         ## open the teaching notebook
    jupyter notebook notebooks/01_walkthrough.ipynb

clean:
    rm -rf results_demo data/raw/synthetic .pytest_cache
    find . -name __pycache__ -type d -prune -exec rm -rf {} +
```

docker-compose.yml

```
services:
  db:
    image: postgis/postgis:16-3.4
    container_name: co2-db
```

```
    environment:
      POSTGRES_USER: co2
      POSTGRES_PASSWORD: co2
      POSTGRES_DB: co2
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

requirements.txt

```
numpy>=1.26
pandas>=2.0
scipy>=1.11
scikit-learn>=1.4
netCDF4>=1.6
pyproj>=3.6
geopandas>=0.14
shapely>=2.0
matplotlib>=3.8
folium>=0.16
sqlalchemy>=2.0
geoalchemy2>=0.14
psycopg2-binary>=2.9
python-dotenv>=1.0
earthaccess>=0.9
requests>=2.31
# development / teaching
pytest>=8.0
jupyter>=1.0
nbformat>=5.9
nbconvert>=7.0
ipykernel>=6.0
```

.env.example

```
# NASA Earthdata login (free: https://urs.earthdata.nasa.gov)
EARTHDATA_USERNAME=
EARTHDATA_PASSWORD=

# PostGIS (matches docker-compose.yml)
DATABASE_URL=postgresql+psycopg2://co2:co2@localhost:5432/co2
```

.gitignore

```
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/
.env
.pytest_cache/

# large or regenerable data
data/raw/oco2/
data/raw/synthetic/
```

```
data/processed/*
!data/processed/.gitkeep

# synthetic demo output must never be committed as if it were a result
results_demo/
```

tests/test_basics.py

```
"""Unit tests for geography, sources and the download helpers (no network, no data files needed)."""
from datetime import date

import numpy as np
import pytest

from highveld_co2.download import cycle_days, date_from_filename, parse_version, pick_latest_version
from highveld_co2.geo import haversine_km, to_lonlat, to_metric
from highveld_co2.sources import SourceIndex, group_complexes, load_sources


# ---------------- geo ----------------
def test_metric_roundtrip():
    lon, lat = np.array([27.5, 29.2, 30.9]), np.array([-27.5, -26.2, -24.6])
    x, y = to_metric(lon, lat)
    lon2, lat2 = to_lonlat(x, y)
    assert np.allclose(lon, lon2, atol=1e-8) and np.allclose(lat, lat2, atol=1e-8)


def test_metric_distance_agrees_with_haversine():
    # 1 degree of latitude and a 1.5 degree east-west step, both inside the study box
    for (lo1, la1, lo2, la2) in [(29.0, -27.0, 29.0, -26.0), (28.0, -26.0, 29.5, -26.0)]:
        x, y = to_metric([lo1, lo2], [la1, la2])
        metric = np.hypot(x[1] - x[0], y[1] - y[0]) / 1000
        assert metric == pytest.approx(haversine_km(lo1, la1, lo2, la2), rel=0.005)


# ---------------- sources ----------------
def test_load_sources_contains_known_plants():
    s = load_sources()
    assert {"Kendal power station", "Majuba power station"} <= set(s["name"])
    assert s["capacity_mw"].isna().sum() >= 1          # Secunda has no capacity figure
    assert s["source_id"].tolist() == list(range(len(s)))


def test_complexes_chain_neighbours_and_order_by_capacity():
    src, comp = group_complexes(load_sources(), eps_km=30)
    assert comp["complex_id"].tolist() == list(range(len(comp)))
    assert comp["total_capacity_mw"].is_monotonic_decreasing
    cid = dict(zip(src["name"], src["complex_id"]))
    assert cid["Matla power station"] == cid["Kriel power station"]      # ~4 km apart
    assert cid["Majuba power station"] != cid["Kendal power station"]    # >100 km apart


def test_source_index_distance_is_zero_at_a_source():
    src, _ = group_complexes(load_sources())
    idx = SourceIndex(src)
    d, i = idx.query(src["x_m"].values, src["y_m"].values)
    assert np.allclose(d, 0)
    assert (i == np.arange(len(src))).all()


# ---------------- download helpers ----------------
```

```
def _g(version):
    return {"umm": {"CollectionReference": {"Version": version}}}


def test_parse_version():
    assert parse_version("11.2r") == 11.2 and parse_version("10r") == 10.0 and parse_version("junk") == -1.0


def test_pick_latest_version_keeps_one_version():
    gr = [_g("10r"), _g("11.1r"), _g("11.2r"), _g("11.2r")]
    kept, used = pick_latest_version(gr)
    assert used == "11.2r" and len(kept) == 2
    kept, used = pick_latest_version(gr, wanted="10r")
    assert used == "10r" and len(kept) == 1
    assert pick_latest_version([]) == ([], None)


def test_cycle_days_repeat_every_16_days():
    days = cycle_days(date(2023, 6, 1), [2, 7], date(2023, 6, 1), date(2023, 7, 31))
    assert days == [date(2023, 6, 3), date(2023, 6, 8), date(2023, 6, 19), date(2023, 6, 24),
                    date(2023, 7, 5), date(2023, 7, 10), date(2023, 7, 21), date(2023, 7, 26)]


def test_date_from_filename():
    assert date_from_filename("oco2_LtCO2_230715_B11014Ar_230925185436s.nc4") == date(2023, 7, 15)
    assert date_from_filename("something_else.nc") is None
```

tests/test_anomaly.py

```
"""Tests for pass segmentation, along-track coordinate and background estimation."""
import numpy as np
import pandas as pd

from highveld_co2.anomaly import MAD_TO_SIGMA, along_track_km, assign_passes, estimate_background


def _times(starts_s, n=50, step=0.3):
    t = np.concatenate([s + np.arange(n) * step for s in starts_s])
    return pd.DataFrame({"t_s": t, "time": pd.to_datetime(t, unit="s", utc=True)})


def test_assign_passes_splits_on_time_gap_and_numbers_within_day():
    base = pd.Timestamp("2023-07-15 10:00", tz="UTC").timestamp()
    df = assign_passes(_times([base, base + 6000, base + 12000]), gap_s=300)
    assert df["pass_id"].nunique() == 3
    assert sorted(df["pass_id"].unique()) == ["2023-07-15_0", "2023-07-15_1", "2023-07-15_2"]


def test_assign_passes_keeps_one_pass_together():
    base = pd.Timestamp("2023-07-15 10:00", tz="UTC").timestamp()
    assert assign_passes(_times([base], n=500), gap_s=300)["pass_id"].nunique() == 1


def test_along_track_recovers_length_and_orientation():
    # a straight 100 km track heading north-west, 8 footprints wide
    u = np.linspace(0, 100_000, 400)
    x = 500_000 - 0.14 * u + np.random.default_rng(0).normal(0, 3000, 400)
    y = 7_000_000 + 0.99 * u
    t = np.linspace(0, 15, 400)
    s = along_track_km(x, y, t)
    assert s.min() == 0
```

```
    assert 95 < s.max() < 105
    assert np.corrcoef(s, t)[0, 1] > 0.99          # oriented along increasing time
    s_rev = along_track_km(x, y, t[::-1])
    assert np.corrcoef(s_rev, t[::-1])[0, 1] > 0.99


def test_running_background_follows_a_gradient_that_the_median_misses():
    rng = np.random.default_rng(1)
    s = np.linspace(0, 400, 2000)
    truth = 410 + 0.01 * s                                  # +4 ppm over the pass
    xco2 = truth + rng.normal(0, 0.6, s.size)
    xco2 = xco2 + np.where((s > 180) & (s < 220), 3.0, 0.0)   # a plume in the middle
    far = ~((s > 150) & (s < 250))                          # plume zone excluded from the background
    bg_run, sig = estimate_background(s, xco2, far, "running", 100.0, 0.3)
    bg_med, _ = estimate_background(s, xco2, far, "median", 100.0, 0.3)
    assert np.sqrt(np.mean((bg_run - truth) ** 2)) < 0.3
    assert np.sqrt(np.mean((bg_med - truth) ** 2)) > 1.0      # the median cannot follow a gradient
    assert 0.5 < sig < 0.8                                  # close to the true noise of 0.6


def test_sigma_is_robust_to_outliers_and_floored():
    rng = np.random.default_rng(2)
    s = np.linspace(0, 300, 1500)
    x = 415 + rng.normal(0, 0.6, s.size)
    x[:30] += 15.0                                          # 2 % wild outliers
    far = np.ones(s.size, bool)
    _, sig = estimate_background(s, x, far, "median", 100.0, 0.3)
    assert 0.5 < sig < 0.75                                 # a plain std would be ~2.5
    _, floored = estimate_background(s, np.full(s.size, 415.0), far, "median", 100.0, 0.3)
    assert floored == 0.3
    assert MAD_TO_SIGMA == 1.4826
```

tests/test_events_stats.py

```
"""Tests for binning, event merging, attribution statistics and compositing, on hand-built data."""
import numpy as np
import pandas as pd
import pytest

from highveld_co2.attribute import control_test, link_events, wilson_ci
from highveld_co2.composite import radial_profile
from highveld_co2.config import Params
from highveld_co2.events import bin_passes, group_events
from highveld_co2.sources import SourceIndex

P = Params()


def _index():
    src = pd.DataFrame({"name": ["Plant A"], "x_m": [500_000.0], "y_m": [7_000_000.0], "complex_id": [0]})
    return SourceIndex(src)


def _soundings(plume_bins=(), amp=2.0, seed=0, n_per_bin=20, n_bins=60):
    """One synthetic pass, 5 km bins, noise 0.7 ppm, optional +amp ppm in the given bins."""
    rng = np.random.default_rng(seed)
    s = np.repeat(np.arange(n_bins) * 5 + 2.5, n_per_bin)
    anom = rng.normal(0, 0.7, s.size)
    in_plume = np.isin(np.floor(s / 5).astype(int), list(plume_bins))
    anom = anom + np.where(in_plume, amp, 0.0)
    return pd.DataFrame({
```

```
        "pass_id": "p0", "s_km": s, "anomaly_ppm": anom, "sigma_ppm": 0.7, "xco2": 415 + anom,
        "lat": -26.0, "lon": 28.0, "x_m": 500_000.0 + s * 1000, "y_m": 7_000_000.0,
        "time": pd.Timestamp("2023-07-15 11:30", tz="UTC"),
    })


def test_a_plume_becomes_exactly_one_event():
    bins = bin_passes(_soundings(plume_bins=range(20, 24)), _index(), P)
    ev = group_events(bins, P)
    assert len(ev) == 1
    assert ev["n_bins"].iloc[0] == 4 and ev["length_km"].iloc[0] == 20
    assert 1.5 < ev["peak_anomaly_ppm"].iloc[0] < 2.6


def test_two_separated_plumes_make_two_events():
    bins = bin_passes(_soundings(plume_bins=[10, 11, 40, 41]), _index(), P)
    assert len(group_events(bins, P)) == 2


def test_pure_noise_gives_almost_no_events():
    flagged = sum(bin_passes(_soundings(seed=s), _index(), P)["flagged"].sum() for s in range(30))
    assert flagged <= 3        # 1800 bins at z>=3 plus >=1 ppm: essentially none


def test_bins_below_minimum_count_are_dropped():
    df = _soundings(n_per_bin=3)
    assert bin_passes(df, _index(), P).empty


def test_link_events_attributes_by_distance():
    ev = pd.DataFrame({"x_m": [500_000 + 10_000, 500_000 + 100_000], "y_m": [7_000_000.0] * 2})
    out = link_events(ev, _index(), P)
    assert out["attributed"].tolist() == [True, False]
    assert out["nearest_source"].iloc[0] == "Plant A"


def test_wilson_ci_known_values():
    lo, hi = wilson_ci(0, 10)
    assert lo == 0.0 and hi == pytest.approx(0.2775, abs=1e-3)
    lo, hi = wilson_ci(5, 10)
    assert (lo, hi) == (pytest.approx(0.2366, abs=1e-3), pytest.approx(0.7634, abs=1e-3))
    assert all(np.isnan(wilson_ci(0, 0)))


def _bins_for_control(rng, n_pass=40):
    rows = []
    for p in range(n_pass):
        for _ in range(10):      # near bins: 20 % flagged
            rows.append({"pass_id": f"p{p}", "dist_source_km": rng.uniform(0, 25), "flagged": rng.random() <
0.20})
        for _ in range(40):      # far bins: 2 % flagged
            rows.append({"pass_id": f"p{p}", "dist_source_km": rng.uniform(70, 150), "flagged": rng.random() <
0.02})
    return pd.DataFrame(rows)


def test_control_test_detects_enrichment_and_gives_a_bootstrap_interval():
    c = control_test(_bins_for_control(np.random.default_rng(0)), P, n_boot=500)
    assert c["near_rate"] > 5 * c["far_rate"]
    assert c["fisher_p"] < 1e-6
    lo, hi = c["rate_ratio_ci"]
    assert lo > 1 and lo < c["rate_ratio"] < hi
```

```
def test_control_test_is_calibrated_when_there_is_no_enrichment():
    """With the same flag rate everywhere, p < 0.01 should be rare (about 1 run in 100).

    A single seed would make this test fail by chance now and then, so run 20 and allow a few.
    (A separate 400-run check gave 0.7 % below 0.01 and 3.5 % below 0.05.)
    """
    small_p = 0
    for seed in range(20):
        rng = np.random.default_rng(1000 + seed)
        b = _bins_for_control(rng)
        b["flagged"] = rng.random(len(b)) < 0.05
        c = control_test(b, P, n_boot=20)
        assert 0.3 < c["rate_ratio"] < 3.5
        small_p += c["fisher_p"] < 0.01
    assert small_p <= 3


def test_radial_profile_recovers_a_decaying_signal():
    rng = np.random.default_rng(3)
    d = rng.uniform(0, 100, 6000)
    bins = pd.DataFrame({"pass_id": rng.integers(0, 60, d.size).astype(str), "dist_source_km": d,
                         "mean_anomaly_ppm": 1.0 * np.exp(-d / 20) + rng.normal(0, 0.2, d.size)})
    prof = radial_profile(bins, edges_km=(0, 10, 30, 60, 100), n_boot=300)
    m = prof["mean_anomaly_ppm"].values
    assert (np.diff(m) < 0).all()                       # monotonically decreasing
    assert prof["ci_low"].iloc[0] < m[0] < prof["ci_high"].iloc[0]
    assert abs(m[-1]) < 0.1
```

tests/test_pipeline.py

```
"""Integration tests: synthetic files -> reader -> full pipeline (-> optional PostGIS)."""
import os

import numpy as np
import pytest
from netCDF4 import Dataset

from highveld_co2.pipeline import run
from highveld_co2.qc import apply_qc
from highveld_co2.read_oco2 import read_lite_file, read_many
from highveld_co2.synthetic import make_files


@pytest.fixture(scope="module")
def synth_dir(tmp_path_factory):
    d = tmp_path_factory.mktemp("synthetic")
    make_files(d, start="2021-06-01", n_days=400, seed=3)
    return d


def test_reader_returns_box_soundings_and_flags_synthetic(synth_dir):
    df = read_many(sorted(synth_dir.glob("*.nc4")))
    assert df.attrs["synthetic"] is True
    assert df["lon"].between(27, 31).all() and df["lat"].between(-28, -24.5).all()
    assert {"xco2", "xco2_unc", "qf", "t_s", "time", "op_mode", "land_water"} <= set(df.columns)


def test_qc_removes_flagged_soundings_and_reports(synth_dir):
    df = read_many(sorted(synth_dir.glob("*.nc4")))
```

```
    clean, rep = apply_qc(df)
    assert (clean["qf"] == 0).all()
    assert rep["n_out"] == len(clean) < rep["n_in"]
    assert rep["removed_quality_flag"] > 0


def test_qc_skips_rules_whose_columns_are_missing(synth_dir):
    df = read_many(sorted(synth_dir.glob("*.nc4")))
    df["op_mode"] = np.nan
    df["land_water"] = np.nan
    _, rep = apply_qc(df)
    assert "skipped" in rep["operation_mode_rule"] and "skipped" in rep["land_water_rule"]


def test_missing_required_variable_gives_actionable_error(tmp_path):
    p = tmp_path / "bad.nc4"
    with Dataset(p, "w") as ds:
        ds.createDimension("n", 3)
        ds.createVariable("latitude", "f8", ("n",))[:] = [1, 2, 3]
    with pytest.raises(KeyError, match="inspect_file"):
        read_lite_file(p)


def test_refuses_to_mix_synthetic_and_real(synth_dir, tmp_path):
    src = sorted(synth_dir.glob("*.nc4"))[0]
    real_like = tmp_path / "oco2_LtCO2_230101_B11014Ar_real.nc4"
    with Dataset(src) as a, Dataset(real_like, "w") as b:       # copy without the synthetic attribute
        b.createDimension("sounding_id", a.dimensions["sounding_id"].size)
        for name, v in a.variables.items():
            nv = b.createVariable(name, v.dtype, v.dimensions)
            nv[:] = v[:]
        sg = b.createGroup("Sounding")
        for name, v in a.groups["Sounding"].variables.items():
            nv = sg.createVariable(name, v.dtype, v.dimensions)
            nv[:] = v[:]
    with pytest.raises(ValueError, match="mix"):
        read_many([src, real_like])


def test_pipeline_end_to_end(synth_dir, tmp_path):
    out = tmp_path / "out"
    r = run(synth_dir, out, make_figures=True, verbose=False)
    assert r["summary"]["synthetic_data"] is True
    assert r["summary"]["n_valid_overpasses"] > 30
    for f in ("summary.json", "events.csv", "bins.csv.gz", "complex_stats.csv", "fig_anomaly_map.png",
              "fig_radial_profile.png", "fig_example_pass.png", "fig_control_test.png", "map.html"):
        assert (out / f).exists(), f
    c = r["control"]
    assert c["near_rate"] > 3 * c["far_rate"]                  # the injected plumes are found near sources
    assert c["fisher_p"] < 1e-6
    # events must sit near sources MORE than chance: compare with how close the average bin is
    baseline = (r["bins"]["dist_source_km"] <= r["params"].attrib_km).mean()
    assert r["events"]["attributed"].mean() > 1.5 * baseline
    top = r["hotspots"].iloc[0]
    assert top["explained_by_source"]                          # the busiest hotspot is the coal cluster
    assert r["profile"]["mean_anomaly_ppm"].iloc[0] > r["profile"]["mean_anomaly_ppm"].iloc[-1]


def test_pipeline_is_deterministic(synth_dir, tmp_path):
    a = run(synth_dir, tmp_path / "a", make_figures=False, verbose=False)
    b = run(synth_dir, tmp_path / "b", make_figures=False, verbose=False)
    assert a["summary"]["n_events"] == b["summary"]["n_events"]
```

```
    assert a["control"]["near_flagged"] == b["control"]["near_flagged"]


@pytest.mark.skipif(not os.getenv("TEST_DATABASE_URL"),
                    reason="set TEST_DATABASE_URL to a THROWAWAY PostGIS database to run (tables are truncated)")
def test_postgis_roundtrip(synth_dir, tmp_path):
    from sqlalchemy import create_engine, text

    from highveld_co2.db import save_all
    r = run(synth_dir, tmp_path / "db", make_figures=False, verbose=False)
    eng = create_engine(os.environ["TEST_DATABASE_URL"])
    counts = save_all(r["sources"], r["complexes"], r["passes"], r["bins"], r["events"], r["cstats"], engine=eng)
    assert counts["events"] == len(r["events"]) and counts["bins"] == len(r["bins"])
    with eng.connect() as con:
        nan_rows = con.execute(text("SELECT COUNT(*) FROM complex_stats WHERE frequency =
'NaN'::float8")).scalar()
    assert nan_rows == 0                                        # NaN must be stored as NULL

    # the SQL control test must reproduce the Python one exactly (regression: it once used a stale 30 km)
    from highveld_co2.db import run_query
    sql = run_query("control_test", engine=eng).set_index("zone")
    c = r["control"]
    assert (int(sql.loc["near", "bins"]), int(sql.loc["near", "flagged"])) == (c["near_bins"], c["near_flagged"])
    assert (int(sql.loc["far", "bins"]), int(sql.loc["far", "flagged"])) == (c["far_bins"], c["far_flagged"])
```